# N01 - INCREMENTALDBSCAN - CODE - ĐẠO VĂN

*by* Anh 31241020523 - Lý Huỳnh Phương

**Submission date:** 06-Nov-2026 12:15PM (UTC+0700)
**Submission ID:** 3081635468
**File name:** 53983_Anh_31241020523_-_L__Hu_nh_Ph_ng_N01_-_INCREMENTALDBSCAN_-_CODE_-__O_V_N_634_1630116339.ipynb (173.15K)
**Word count:** 17839
**Character count:** 121426

```
{
 "cells": [
  {
   "cell_type": "markdown",
   "id": "454302ca",
   "metadata": {},
   "source": [
    "### TỔNG HỢP CODE TỪ CÁC FILE .PY ĐỂ CHECK ĐẠO VĂN"
   ]
  },
  {
   "cell_type": "markdown",
   "id": "97614b52",
   "metadata": {},
   "source": [
    "#### benchmark_density_algorithms.py"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": null,
   "id": "056add14",
   "metadata": {
    "vscode": {
     "languageId": "plaintext"
    }
   },
   "outputs": [],
   "source": [
    "\"\"\"So sánh Incremental DBSCAN OOP với ba thuật toán density-based
của sklearn.\n",
    "\n",
    "Chạy từ gốc repo:\n",
    "    python -m baseline.benchmark_density_algorithms --repeats 5 --
max-steps 20\n",
    "\n",
    "Mỗi ca 2D/3D/4D lấy số chiều đầu từ cùng stream thẻ; mọi phương pháp
trong một ca\n",
    "nhận cùng D0, thứ tự cập nhật, tọa độ, Euclidean và MinPts.\n",
    "Incremental DBSCAN cập nhật cục bộ; sklearn DBSCAN, HDBSCAN và
OPTICS được fit\n",
    "lại toàn bộ sau mỗi cập nhật vì các estimator này không có giao diện
insert/delete.\n",
    "\"\"\"\n",
    "\n",
    "from __future__ import annotations\n",
    "\n",
    "import argparse\n",
    "import inspect\n",
    "import json\n",
    "import platform\n",
    "import random\n",
    "import time\n",
    "from pathlib import Path\n",
```

```
    "from typing import Callable\n",
    "\n",
    "import matplotlib\n",
    "import numpy as np\n",
    "import pandas as pd\n",
    "import sklearn\n",
    "from sklearn.cluster import DBSCAN as SklearnDBSCAN\n",
    "from sklearn.cluster import HDBSCAN, OPTICS\n",
    "from sklearn.metrics import adjusted_rand_score\n",
    "\n",
    "matplotlib.use(\"Agg\")\n",
    "\n",
    "from src.experiments.engine import IncrementalEngine\n",
    "from src.preprocessing.stream_scaling import stream_coordinates\n",
    "from src.visualization.plotter import report_style\n",
    "from .sklearn_dbscan import IdMap, read_stream\n",
    "\n",
    "ROOT = Path(__file__).resolve().parents[1]\n",
    "RAW = ROOT / \"data\" / \"raw\"\n",
    "RESULTS = ROOT / \"results\"\n",
    "FIGURES = ROOT / \"report\" / \"figures\"\n",
    "RUNS_CSV = RESULTS / \"density_algorithm_benchmark_runs.csv\"\n",
    "REPEAT_RESULTS_CSV = RESULTS /
\"density_algorithm_benchmark_repeats.csv\"\n",
    "SUMMARY_CSV = RESULTS /
\"density_algorithm_benchmark_summary.csv\"\n",
    "README_TXT = RESULTS /
\"density_algorithm_benchmark_README.txt\"\n",
    "FIGURE_PATH = FIGURES / \"density_algorithm_benchmark.png\"\n",
    "\n",
    "CARD_STREAM = RAW / \"card_stream\"\n",
    "STREAMS = {\n",
    "    \"Thẻ 2D\": (CARD_STREAM, 2),\n",
    "    \"Thẻ 3D\": (CARD_STREAM, 3),\n",
    "    \"Thẻ 4D\": (CARD_STREAM, 4),\n",
    "}\n",
    "METHODS = (\n",
    "    \"Incremental DBSCAN (OOP)\",\n",
    "    \"sklearn DBSCAN (re-fit)\",\n",
    "    \"sklearn HDBSCAN (re-fit)\",\n",
    "    \"sklearn OPTICS (re-fit)\",\n",
    ")\n",
    "METHOD_COLORS = (\"#1f77b4\", \"#7f7f7f\", \"#9467bd\",
\"#2ca02c\")\n",
    "\n",
    "\n",
    "def _make_sklearn_estimator(method: str, eps: float, min_pts:
int):\n",
    "    \"\"\"Tạo estimator tĩnh với cấu hình cố định cho
benchmark.\"\"\"\n",
    "    if method == \"sklearn DBSCAN (re-fit)\":\n",
    "        return SklearnDBSCAN(eps=eps, min_samples=min_pts,
metric=\"euclidean\", n_jobs=1)\n",
    "    if method == \"sklearn HDBSCAN (re-fit)\":\n",
```

```
    "        options = {\n",
    "            \"min_cluster_size\": max(2, min_pts),\n",
    "            \"min_samples\": min_pts,\n",
    "            \"metric\": \"euclidean\",\n",
    "            \"cluster_selection_method\": \"eom\",\n",
    "            \"n_jobs\": 1,\n",
    "        }\n",
    "\n",
    "        if \"copy\" in inspect.signature(HDBSCAN).parameters:\n",
    "            options[\"copy\"] = True\n",
    "        return HDBSCAN(**options)\n",
    "    if method == \"sklearn OPTICS (re-fit)\":\n",
    "        return OPTICS(\n",
    "            min_samples=min_pts,\n",
    "            max_eps=eps + 1e-6,\n",
    "            metric=\"euclidean\",\n",
    "            cluster_method=\"dbscan\",\n",
    "            eps=eps,\n",
    "            n_jobs=1,\n",
    "        )\n",
    "    raise ValueError(f\"Không phải estimator sklearn:
{method}\")\n",
    "\n",
    "\n",
    "def _parameters(method: str, eps: float, min_pts: int, features:
list[str]) -> str:\n",
    "    \"\"\"Lưu đầy đủ tham số để đọc kết quả mà không phải suy ra từ
mã nguồn.\"\"\"\n",
    "    common = {\"dimensions\": len(features), \"features\": features,
\"metric\": \"euclidean\"}\n",
    "    if method == \"Incremental DBSCAN (OOP)\":\n",
    "        config = {**common, \"eps\": eps, \"min_pts\": min_pts,
\"update\": \"local insert/delete\"}\n",
    "    elif method == \"sklearn DBSCAN (re-fit)\":\n",
    "        config = {**common, \"eps\": eps, \"min_samples\": min_pts,
\"update\": \"full refit\"}\n",
    "    elif method == \"sklearn HDBSCAN (re-fit)\":\n",
    "        config = {\n",
    "            **common,\n",
    "            \"min_cluster_size\": max(2, min_pts),\n",
    "            \"min_samples\": min_pts,\n",
    "            \"cluster_selection_method\": \"eom\",\n",
    "            \"n_jobs\": 1,\n",
    "            \"update\": \"full refit\",\n",
    "        }\n",
    "        if \"copy\" in inspect.signature(HDBSCAN).parameters:\n",
    "            config[\"copy\"] = True\n",
    "    else:\n",
    "        config = {\n",
    "            **common,\n",
    "            \"min_samples\": min_pts,\n",
    "            \"eps\": eps,\n",
    "            \"max_eps\": eps + 1e-6,\n",
    "            \"cluster_method\": \"dbscan\",\n",
```

```
    "            \"n_jobs\": 1,\n",
    "            \"update\": \"full refit\",\n",
    "        }\n",
    "    return json.dumps(config, ensure_ascii=False,
sort_keys=True)\n",
    "\n",
    "\n",
    "class _FullRefitDensityModel:\n",
    "    \"\"\"Adapter insert/delete dùng để đo estimator sklearn chạy
lại toàn bộ dữ liệu.\"\"\"\n",
    "\n",
    "    def __init__(self, method: str, eps: float, min_pts: int) ->
None:\n",
    "        self.method = method\n",
    "        self.eps = eps\n",
    "        self.min_pts = min_pts\n",
    "        self.points: dict[int, np.ndarray] = {}\n",
    "        self.labels = np.empty(0, dtype=int)\n",
    "        self.next_id = 0\n",
    "\n",
    "    def fit(self, data: np.ndarray):\n",
    "        self.points = {i: np.asarray(row, dtype=float).copy() for i,
row in enumerate(data)}\n",
    "        self.next_id = len(self.points)\n",
    "        self._refit()\n",
    "        return self\n",
    "\n",
    "    def insert(self, coords) -> int:\n",
    "        point_id = self.next_id\n",
    "        self.next_id += 1\n",
    "        self.points[point_id] = np.asarray(coords,
dtype=float).copy()\n",
    "        self._refit()\n",
    "        return point_id\n",
    "\n",
    "    def delete(self, point_id: int) -> None:\n",
    "        if point_id not in self.points:\n",
    "            raise ValueError(f\"Không tìm thấy
point_id={point_id}\")\n",
    "        del self.points[point_id]\n",
    "        self._refit()\n",
    "\n",
    "    def _refit(self) -> None:\n",
    "        if not self.points:\n",
    "            self.labels = np.empty(0, dtype=int)\n",
    "            return\n",
    "        data = np.vstack(list(self.points.values()))\n",
    "        estimator = _make_sklearn_estimator(self.method, self.eps,
self.min_pts)\n",
    "        estimator.fit(data)\n",
    "        self.labels = np.asarray(estimator.labels_, dtype=int)\n",
    "\n",
    "    def snapshot(self) -> dict:\n",
    "        ids = list(self.points)\n",
```

```
    "        dimension = len(next(iter(self.points.values()))) if
self.points else 0\n",
    "        coords = np.vstack([self.points[point_id] for point_id in
ids]) if ids else np.empty((0, dimension))\n",
    "        return {\"ids\": np.asarray(ids, dtype=int), \"coords\":
coords, \"labels\": self.labels.copy()}\n",
    "\n",
    "\n",
    "def _new_model(method: str, eps: float, min_pts: int):\n",
    "    if method == \"Incremental DBSCAN (OOP)\":\n",
    "        return IncrementalEngine(eps, min_pts)\n",
    "    return _FullRefitDensityModel(method, eps, min_pts)\n",
    "\n",
    "\n",
    "def _snapshot_state(model, ids: IdMap) -> tuple[dict[int, int], int,
int, float]:\n",
    "    \"\"\"Trả labels theo point_id của stream cùng số cụm và tỷ lệ
noise.\"\"\"\n",
    "    snap = model.snapshot()\n",
    "    state = {\n",
    "        ids.stream_id(engine_id): int(label)\n",
    "        for engine_id, label in zip(snap[\"ids\"].tolist(),
snap[\"labels\"].tolist())\n",
    "    }\n",
    "    n_noise = sum(label == -1 for label in state.values())\n",
    "    n_clusters = len(set(state.values()) - {-1})\n",
    "    noise_fraction = n_noise / len(state) if state else 0.0\n",
    "    return state, n_clusters, n_noise, noise_fraction\n",
    "\n",
    "\n",
    "def _run_one_stream(\n",
    "    method: str,\n",
    "    stream_name: str,\n",
    "    d0: pd.DataFrame,\n",
    "    updates: pd.DataFrame,\n",
    "    feature_cols: list[str],\n",
    "    eps: float,\n",
    "    min_pts: int,\n",
    "    initial_coords: np.ndarray,\n",
    "    update_coords: np.ndarray,\n",
    ") -> tuple[list[dict], dict[int, dict[int, int]]]:\n",
    "    model = _new_model(method, eps, min_pts)\n",
    "    ids = IdMap()\n",
    "    records: list[dict] = []\n",
    "    states: dict[int, dict[int, int]] = {}\n",
    "\n",
    "    started = time.perf_counter()\n",
    "    model.fit(initial_coords)\n",
    "    initial_ms = (time.perf_counter() - started) * 1000\n",
    "    for stream_id, engine_id in zip(d0[\"point_id\"].tolist(),
model.snapshot()[\"ids\"].tolist()):\n",
    "        ids.bind(stream_id, engine_id)\n",
    "\n",
```

```
    "    def record(step: int, op: str, point_id, case: str, elapsed_ms:
float, n_before: int) -> None:\n",
    "        state, n_clusters, n_noise, noise_fraction =
_snapshot_state(model, ids)\n",
    "        states[step] = state\n",
    "        records.append({\n",
    "            \"stream\": stream_name,\n",
    "            \"method\": method,\n",
    "            \"step\": step,\n",
    "            \"op\": op,\n",
    "            \"point_id\": point_id,\n",
    "            \"case\": case,\n",
    "            \"n_before\": n_before,\n",
    "            \"n_after\": len(state),\n",
    "            \"dimensions\": int(initial_coords.shape[1]),\n",
    "            \"features\": \",\".join(feature_cols),\n",
    "            \"n_clusters\": n_clusters,\n",
    "            \"n_noise\": n_noise,\n",
    "            \"noise_fraction\": noise_fraction,\n",
    "            \"elapsed_ms\": elapsed_ms,\n",
    "            \"eps\": eps,\n",
    "            \"min_pts_reference\": min_pts,\n",
    "            \"parameters\": _parameters(method, eps, min_pts,
feature_cols),\n",
    "        })\n",
    "\n",
    "    record(0, \"fit\", None, \"\", initial_ms, 0)\n",
    "    for index, (_, row) in enumerate(updates.iterrows()):\n",
    "        step = int(row[\"step\"])\n",
    "        stream_id = int(row[\"point_id\"])\n",
    "        op = str(row[\"op\"])\n",
    "        case_value = row.get(\"verified_case\", \"\")\n",
    "        case = \"\" if pd.isna(case_value) else str(case_value)\n",
    "        n_before = len(model.snapshot()[\"ids\"])\n",
    "        if op == \"insert\":\n",
    "            started = time.perf_counter()\n",
    "            engine_id = model.insert(update_coords[index])\n",
    "            elapsed_ms = (time.perf_counter() - started) * 1000\n",
    "            ids.bind(stream_id, engine_id)\n",
    "        elif op == \"delete\":\n",
    "            engine_id = ids.release(stream_id)\n",
    "            started = time.perf_counter()\n",
    "            model.delete(engine_id)\n",
    "            elapsed_ms = (time.perf_counter() - started) * 1000\n",
    "        else:\n",
    "            raise ValueError(f\"Toán tử cập nhật không hợp lệ ở step
{step}: {op}\")\n",
    "        record(step, op, stream_id, case, elapsed_ms, n_before)\n",
    "    return records, states\n",
    "\n",
    "\n",
    "def _add_agreement_scores(records: list[dict], states_by_method:
dict[str, dict[int, dict[int, int]]]) -> None:\n",
    "    reference = states_by_method[\"Incremental DBSCAN (OOP)\"]\n",
```

```
    "    methods = list(states_by_method)\n",
    "    score_by_method_step = {}\n",
    "    for method in methods:\n",
    "        for step, state in states_by_method[method].items():\n",
    "            ref = reference[step]\n",
    "            if set(ref) != set(state):\n",
    "                raise RuntimeError(f\"Tập điểm khác nhau ở {method},
step={step}\")\n",
    "            ordered_ids = sorted(ref)\n",
    "            ref_labels = [ref[point_id] for point_id in
ordered_ids]\n",
    "            labels = [state[point_id] for point_id in
ordered_ids]\n",
    "            ari = float(adjusted_rand_score(ref_labels, labels))\n",
    "            noise_equal = {i for i, label in ref.items() if label ==
-1} == {i for i, label in state.items() if label == -1}\n",
    "            score_by_method_step[(method, step)] = (ari,
noise_equal)\n",
    "    for row in records:\n",
    "        row[\"ari_vs_incremental\"] =
score_by_method_step[(row[\"method\"], row[\"step\"])][0]\n",
    "        row[\"noise_matches_incremental\"] =
score_by_method_step[(row[\"method\"], row[\"step\"])][1]\n",
    "\n",
    "\n",
    "def _load_prefix(stream_dir: Path, dimensions: int, max_steps:
int):\n",
    "    d0, updates, params, feature_cols = read_stream(stream_dir)\n",
    "    if not 1 <= dimensions <= len(feature_cols):\n",
    "        raise ValueError(f\"Số chiều {dimensions} không hợp lệ cho
{len(feature_cols)} đặc trưng.\")\n",
    "    updates =
updates.sort_values(\"step\").head(max_steps).reset_index(drop=True)\n",
    "    initial_coords = stream_coordinates(params, d0, feature_cols)[:,
:dimensions]\n",
    "    update_coords = stream_coordinates(params, updates,
feature_cols)[:, :dimensions]\n",
    "    frozen_scaling = params.get(\"frozen_scaling\")\n",
    "    scaled_features = frozen_scaling[\"columns\"] if frozen_scaling
else feature_cols\n",
    "    return d0, updates, params, scaled_features[:dimensions],
initial_coords, update_coords\n",
    "\n",
    "\n",
    "def run_benchmark(\n",
    "    *,\n",
    "    stream_names: list[str] | None = None,\n",
    "    repeats: int = 5,\n",
    "    warmups: int = 1,\n",
    "    max_steps: int = 20,\n",
    "    seed: int = 42,\n",
    "    log: Callable[[str], None] = print,\n",
    ") -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:\n",
```

```
    "    \"\"\"Benchmark các prefix của cùng stream thẻ ở 2D/3D/4D;
sklearn refit toàn bộ sau từng update.\"\"\"\n",
    "    if repeats < 2:\n",
    "        raise ValueError(\"repeats phải từ 2 trở lên\")\n",
    "    if warmups < 0:\n",
    "        raise ValueError(\"warmups không được âm\")\n",
    "    if max_steps < 1:\n",
    "        raise ValueError(\"max_steps phải lớn hơn 0\")\n",
    "    chosen = [name for name in STREAMS if stream_names is None or
name in stream_names]\n",
    "    unknown = set(stream_names or ()) - set(STREAMS)\n",
    "    if unknown:\n",
    "        raise ValueError(f\"Không tìm thấy stream: {',
'.join(sorted(unknown))}\")\n",
    "\n",
    "    all_records = []\n",
    "    rng = random.Random(seed)\n",
    "    for stream_name in chosen:\n",
    "        stream_dir, dimensions = STREAMS[stream_name]\n",
    "        d0, updates, params, feature_cols, initial_coords,
update_coords = _load_prefix(\n",
    "            stream_dir, dimensions, max_steps\n",
    "        )\n",
    "        eps = float(params.get(\"eps\", params.get(\"eps_km\")))\n",
    "        min_pts = int(params[\"minpts\"])\n",
    "\n",
    "        # Warm-up chỉ fit D0; các phép đo bên dưới vẫn phát lại từng
chuỗi từ đầu.\n",
    "        for method in METHODS:\n",
    "            for _ in range(warmups):\n",
    "                _new_model(method, eps,
min_pts).fit(initial_coords)\n",
    "\n",
    "        stream_records = []\n",
    "        for repeat in range(1, repeats + 1):\n",
    "            order = list(METHODS)\n",
    "            rng.shuffle(order)\n",
    "            states_by_method = {}\n",
    "            repeat_records = []\n",
    "            for method in order:\n",
    "                records, states = _run_one_stream(\n",
    "                    method, stream_name, d0, updates, feature_cols,
eps, min_pts,\n",
    "                    initial_coords, update_coords,\n",
    "                )\n",
    "                for row in records:\n",
    "                    row[\"repeat\"] = repeat\n",
    "                repeat_records.extend(records)\n",
    "                states_by_method[method] = states\n",
    "            _add_agreement_scores(repeat_records,
states_by_method)\n",
    "            stream_records.extend(repeat_records)\n",
    "            stream_records.sort(key=lambda row:
(METHODS.index(row[\"method\"]), row[\"repeat\"], row[\"step\"]))\n",
```

```
    "            updates_count = len(updates)\n",
    "            log(\n",
    "                f\"{stream_name}: hoàn tất lượt {repeat}/{repeats},
\"\n",
    "                f\"{updates_count} update, Eps={eps:g},
MinPts={min_pts}\"\n",
    "            )\n",
    "        all_records.extend(stream_records)\n",
    "\n",
    "    runs = pd.DataFrame(all_records)\n",
    "    repeat_results = summarize_repeats(runs)\n",
    "    summary = summarize_runs(repeat_results,
max_steps=max_steps)\n",
    "    return runs, repeat_results, summary\n",
    "\n",
    "\n",
    "def summarize_repeats(runs: pd.DataFrame) -> pd.DataFrame:\n",
    "    \"\"\"Tạo một dòng cho từng stream/phương pháp/lượt để so sánh
trực tiếp.\"\"\"\n",
    "    rows = []\n",
    "    for (stream_name, method, repeat), part in
runs.groupby([\"stream\", \"method\", \"repeat\"], sort=False):\n",
    "        initial = part[part[\"op\"] == \"fit\"].iloc[0]\n",
    "        updates = part[part[\"op\"].isin((\"insert\",
\"delete\"))]\n",
    "        inserts = updates[updates[\"op\"] ==
\"insert\"][\"elapsed_ms\"]\n",
    "        deletes = updates[updates[\"op\"] ==
\"delete\"][\"elapsed_ms\"]\n",
    "        final = part.sort_values(\"step\").iloc[-1]\n",
    "        rows.append({\n",
    "            \"stream\": stream_name,\n",
    "            \"method\": method,\n",
    "            \"repeat\": int(repeat),\n",
    "            \"n_d0\": int(initial[\"n_after\"]),\n",
    "            \"dimensions\": int(initial[\"dimensions\"]),\n",
    "            \"features\": initial[\"features\"],\n",
    "            \"n_updates\": int(len(updates)),\n",
    "            \"n_insert\": int((updates[\"op\"] ==
\"insert\").sum()),\n",
    "            \"n_delete\": int((updates[\"op\"] ==
\"delete\").sum()),\n",
    "            \"eps\": float(initial[\"eps\"]),\n",
    "            \"min_pts_reference\":
int(initial[\"min_pts_reference\"]),\n",
    "            \"parameters\": initial[\"parameters\"],\n",
    "            \"initial_fit_ms\": float(initial[\"elapsed_ms\"]),\n",
    "            \"insert_total_ms\": float(inserts.sum()),\n",
    "            \"insert_median_ms_per_update\": float(inserts.median())
if len(inserts) else 0.0,\n",
    "            \"delete_total_ms\": float(deletes.sum()),\n",
    "            \"delete_median_ms_per_update\": float(deletes.median())
if len(deletes) else 0.0,\n",
```

```
    "            \"update_total_ms\":
float(updates[\"elapsed_ms\"].sum()),\n",
    "            \"ari_vs_incremental_mean\":
float(updates[\"ari_vs_incremental\"].mean()) if len(updates) else
1.0,\n",
    "            \"ari_vs_incremental_final\":
float(final[\"ari_vs_incremental\"]),\n",
    "            \"noise_matches_incremental_fraction\":
float(updates[\"noise_matches_incremental\"].mean()) if len(updates) else
1.0,\n",
    "            \"final_clusters\": int(final[\"n_clusters\"]),\n",
    "            \"final_noise_fraction\":
float(final[\"noise_fraction\"]),\n",
    "        })\n",
    "    return pd.DataFrame(rows)\n",
    "\n",
    "\n",
    "def summarize_runs(repeats: pd.DataFrame, *, max_steps: int) ->
pd.DataFrame:\n",
    "    \"\"\"Tổng hợp các lượt lặp, giữ median/mean/std và min/max thời
gian cập nhật.\"\"\"\n",
    "    rows = []\n",
    "    for (stream_name, method), part in repeats.groupby([\"stream\",
\"method\"], sort=False):\n",
    "        totals = part[\"update_total_ms\"]\n",
    "        rows.append({\n",
    "            \"stream\": stream_name,\n",
    "            \"method\": method,\n",
    "            \"repeats\": int(part[\"repeat\"].nunique()),\n",
    "            \"n_d0\": int(part[\"n_d0\"].iloc[0]),\n",
    "            \"dimensions\": int(part[\"dimensions\"].iloc[0]),\n",
    "            \"features\": part[\"features\"].iloc[0],\n",
    "            \"max_steps_requested\": max_steps,\n",
    "            \"n_updates_per_repeat\":
int(part[\"n_updates\"].median()),\n",
    "            \"n_insert_per_repeat\":
int(part[\"n_insert\"].median()),\n",
    "            \"n_delete_per_repeat\":
int(part[\"n_delete\"].median()),\n",
    "            \"eps\": float(part[\"eps\"].iloc[0]),\n",
    "            \"min_pts_reference\":
int(part[\"min_pts_reference\"].iloc[0]),\n",
    "            \"parameters\": part[\"parameters\"].iloc[0],\n",
    "            \"initial_fit_median_ms\":
float(part[\"initial_fit_ms\"].median()),\n",
    "            \"insert_median_ms_per_update\":
float(part[\"insert_median_ms_per_update\"].median()),\n",
    "            \"delete_median_ms_per_update\":
float(part[\"delete_median_ms_per_update\"].median()),\n",
    "            \"update_total_median_ms\": float(totals.median()),\n",
    "            \"update_total_mean_ms\": float(totals.mean()),\n",
    "            \"update_total_std_ms\": float(totals.std(ddof=1)) if
len(totals) > 1 else 0.0,\n",
    "            \"update_total_min_ms\": float(totals.min()),\n",
```

```
    "            \"update_total_max_ms\": float(totals.max()),\n",
    "            \"ari_vs_incremental_mean\":
float(part[\"ari_vs_incremental_mean\"].mean()),\n",
    "            \"ari_vs_incremental_final_mean\":
float(part[\"ari_vs_incremental_final\"].mean()),\n",
    "            \"noise_matches_incremental_fraction\":
float(part[\"noise_matches_incremental_fraction\"].mean()),\n",
    "            \"final_clusters_median\":
float(part[\"final_clusters\"].median()),\n",
    "            \"final_noise_fraction_median\":
float(part[\"final_noise_fraction\"].median()),\n",
    "        })\n",
    "    return pd.DataFrame(rows)\n",
    "\n",
    "\n",
    "def plot_summary(summary: pd.DataFrame):\n",
    "    \"\"\"Vẽ median tổng thời gian cập nhật; whisker min-max là độ
biến thiên giữa các lượt.\"\"\"\n",
    "    import matplotlib.pyplot as plt\n",
    "\n",
    "    streams = list(dict.fromkeys(summary[\"stream\"].tolist()))\n",
    "    x = np.arange(len(streams), dtype=float)\n",
    "    width = 0.19\n",
    "    with report_style():\n",
    "        fig, ax = plt.subplots(figsize=(10.2, 4.5))\n",
    "        for method_index, (method, color) in enumerate(zip(METHODS,
METHOD_COLORS)):\n",
    "            rows = summary[summary[\"method\"] ==
method].set_index(\"stream\").reindex(streams)\n",
    "            median =
rows[\"update_total_median_ms\"].to_numpy(dtype=float)\n",
    "            low = median -
rows[\"update_total_min_ms\"].to_numpy(dtype=float)\n",
    "            high =
rows[\"update_total_max_ms\"].to_numpy(dtype=float) - median\n",
    "            positions = x + (method_index - (len(METHODS) - 1) / 2)
* width\n",
    "            ax.bar(positions, median, width, color=color,
label=method)\n",
    "            ax.errorbar(positions, median, yerr=np.vstack((low,
high)), fmt=\"none\", ecolor=\"#333333\", capsize=2, linewidth=0.8)\n",
    "        ax.set_yscale(\"log\")\n",
    "        ax.set_xticks(x)\n",
    "        ax.set_xticklabels(streams, fontsize=8)\n",
    "        ax.set_ylabel(\"Tổng thời gian insert/delete (ms,
log)\")\n",
    "        ax.set_title(\"Incremental DBSCAN OOP so với sklearn fit lại
toàn bộ\\nTrung vị chuỗi và khoảng min-max qua các lượt\", loc=\"left\",
fontsize=10)\n",
    "        ax.legend(frameon=False, fontsize=7.5, loc=\"center left\",
bbox_to_anchor=(1.01, 0.5))\n",
    "        ax.grid(axis=\"y\", alpha=0.2)\n",
    "        fig.tight_layout(rect=(0, 0, 0.78, 1))\n",
    "    return fig\n",
```

```
    "\n",
    "\n",
    "def main(argv=None) -> None:\n",
    "    parser = argparse.ArgumentParser(description=__doc__)\n",
    "    parser.add_argument(\"--streams\", nargs=\"+\",
choices=list(STREAMS), default=None)\n",
    "    parser.add_argument(\"--repeats\", type=int, default=5,
help=\"Số lần phát lại cùng chuỗi (tối thiểu 2).\")\n",
    "    parser.add_argument(\"--warmups\", type=int, default=1,
help=\"Số lần warm-up fit D0, không tính vào kết quả.\")\n",
    "    parser.add_argument(\"--max-steps\", type=int, default=20,
help=\"Độ dài prefix stream dùng trong mỗi lượt.\")\n",
    "    parser.add_argument(\"--seed\", type=int, default=42,
help=\"Seed cố định để xáo thứ tự các thuật toán.\")\n",
    "    args = parser.parse_args(argv)\n",
    "    if args.repeats < 2:\n",
    "        parser.error(\"--repeats phải từ 2 trở lên\")\n",
    "    if args.warmups < 0:\n",
    "        parser.error(\"--warmups không được âm\")\n",
    "    if args.max_steps < 1:\n",
    "        parser.error(\"--max-steps phải lớn hơn 0\")\n",
    "    if args.seed < 0:\n",
    "        parser.error(\"--seed phải là số nguyên không âm\")\n",
    "\n",
    "    RESULTS.mkdir(parents=True, exist_ok=True)\n",
    "    FIGURES.mkdir(parents=True, exist_ok=True)\n",
    "    runs, repeat_results, summary = run_benchmark(\n",
    "        stream_names=args.streams,\n",
    "        repeats=args.repeats,\n",
    "        warmups=args.warmups,\n",
    "        max_steps=args.max_steps,\n",
    "        seed=args.seed,\n",
    "        log=lambda message: print(message, flush=True),\n",
    "    )\n",
    "    runs.to_csv(RUNS_CSV, index=False, encoding=\"utf-8\",
lineterminator=\"\\n\")\n",
    "    repeat_results.to_csv(REPEAT_RESULTS_CSV, index=False,
encoding=\"utf-8\", lineterminator=\"\\n\")\n",
    "    summary.to_csv(SUMMARY_CSV, index=False, encoding=\"utf-8\",
lineterminator=\"\\n\")\n",
    "    from src.visualization.plotter import ClusterVisualizer\n",
    "    ClusterVisualizer.save(plot_summary(summary), FIGURE_PATH)\n",
    "    (README_TXT).write_text(\n",
    "        \"Benchmark chuỗi cập nhật: Incremental DBSCAN OOP so với
DBSCAN, HDBSCAN và OPTICS của sklearn chạy lại toàn bộ.\\n\"\n",
    "        f\"Máy: {platform.platform()}, {platform.processor() or
platform.machine()}, Python {platform.python_version()}, \"\n",
    "        f\"scikit-learn {sklearn.__version__}, numpy
{np.__version__}, pandas {pd.__version__}.\\n\"\n",
    "        f\"Mỗi trường hợp chiều: {args.warmups} warm-up fit D0 và
{args.repeats} lượt phát lại prefix tối đa {args.max_steps} update;
\"\n",
    "        f\"seed xáo thứ tự thuật toán: {args.seed}.\\n\"\n",
```

```
    "        \"Ba trường hợp 2D/3D/4D lấy lần lượt 2, 3, 4 đặc trưng đầu
của cùng stream thẻ đã chuẩn hóa đóng băng; D0 và thứ tự update giống
nhau.\\n\"\n",
    "        \"Danh sách đặc trưng: log10_amount_vnd, hour_of_day; thêm
distance_from_home_km cho 3D; thêm txns_last_7d cho 4D.\\n\"\n",
    "        \"Cả bốn phương pháp trong từng trường hợp nhận cùng D0, tọa
độ, metric Euclidean và MinPts tham chiếu. Incremental DBSCAN chỉ cập
nhật vùng ảnh hưởng;\\n\"\n",
    "        \"ba estimator sklearn không có insert/delete nên adapter
cập nhật tập điểm rồi fit lại toàn bộ ở từng step.\\n\"\n",
    "        \"DBSCAN dùng Eps/MinPts trong params.json; HDBSCAN dùng
min_cluster_size=min_samples=MinPts và EOM;\\n\"\n",
    "        \"OPTICS dùng min_samples=MinPts, eps=Eps và
cluster_method='dbscan'; max_eps=Eps+1e-6 để trích cụm tại cùng
ngưỡng.\\n\"\n",
    "        \"HDBSCAN có tham số phân cấp riêng; OPTICS vẫn tính thứ tự
reachability trước khi trích cụm kiểu DBSCAN.\\n\"\n",
    "        \"CSV repeats có một dòng cho mỗi stream/phương pháp/lượt
chạy; CSV runs lưu từng step/lượt; CSV summary tổng hợp
median/mean/std/min/max thời gian qua các lượt.\\n\"\n",
    "        \"Các CSV có cả cấu hình tham số, số cụm/noise, ARI trung
bình/cuối và tỷ lệ noise khớp Incremental DBSCAN OOP.\\n\"\n",
    "        \"Thời gian phụ thuộc máy; kết quả trên prefix stream không
đại diện toàn bộ 300 bước.\\n\",\n",
    "        encoding=\"utf-8\",\n",
    "        newline=\"\\n\",\n",
    "    )\n",
    "    print(f\"\\nĐã ghi: {RUNS_CSV.relative_to(ROOT)}\")\n",
    "    print(f\"Đã ghi: {REPEAT_RESULTS_CSV.relative_to(ROOT)}\")\n",
    "    print(f\"Đã ghi: {SUMMARY_CSV.relative_to(ROOT)}\")\n",
    "    print(f\"Đã ghi: {FIGURE_PATH.relative_to(ROOT)}\")\n",
    "    print(summary.round(3).to_string(index=False))\n",
    "\n",
    "\n",
    "if __name__ == \"__main__\":\n",
    "    main()\n"
   ]
  },
  {
   "cell_type": "markdown",
   "id": "dcfbbb15",
   "metadata": {},
   "source": [
    "#### sklearn_dbscan.py"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": null,
   "id": "6d1c860f",
   "metadata": {
    "vscode": {
     "languageId": "plaintext"
    }
```

```
   },
   "outputs": [],
   "source": [
    "\"\"\"DBSCAN tham chiếu dùng trong baseline và kiểm tra kết quả
Incremental DBSCAN.\"\"\"\n",
    "import json\n",
    "import time\n",
    "from enum import IntEnum\n",
    "from pathlib import Path\n",
    "from collections.abc import Mapping\n",
    "\n",
    "import numpy as np\n",
    "import pandas as pd\n",
    "from sklearn.cluster import DBSCAN\n",
    "from sklearn.metrics import adjusted_rand_score\n",
    "\n",
    "from src.preprocessing.stream_scaling import feature_columns\n",
    "\n",
    "\n",
    "class PointType(IntEnum):\n",
    "    \"\"\"Bản sao PointType của src/core/data_point.py để baseline
không phụ thuộc src/core (so với enum của nhóm theo tên).\"\"\"\n",
    "    UNCLASSIFIED = 0\n",
    "    NOISE = 1\n",
    "    BORDER = 2\n",
    "    CORE = 3\n",
    "\n",
    "\n",
    "# ---------------------------------------------------------------------
(a) DBSCAN đầy đủ\n",
    "def fit_full_dbscan(coords, eps, min_pts):\n",
    "    \"\"\"DBSCAN của sklearn (min_samples = min_pts, lân cận gồm cả
chính điểm).\n",
    "\n",
    "    Trả (labels, point_types): labels là mảng int, -1 là nhiễu, cụm
đánh số từ 0;\n",
    "    point_types là mảng int theo PointType (CORE từ
core_sample_indices_, NOISE khi nhãn -1, còn lại BORDER).\n",
    "    Mảng rỗng trả hai mảng rỗng.\n",
    "    \"\"\"\n",
    "    coords = np.asarray(coords, dtype=float)\n",
    "    if len(coords) == 0:\n",
    "        return np.empty(0, dtype=int), np.empty(0, dtype=int)\n",
    "    model = DBSCAN(eps=eps, min_samples=min_pts).fit(coords)\n",
    "    labels = model.labels_.astype(int)\n",
    "    types = np.where(labels == -1, int(PointType.NOISE),
int(PointType.BORDER))\n",
    "    types[model.core_sample_indices_] = int(PointType.CORE)\n",
    "    return labels, types\n",
    "\n",
    "\n",
    "#adapter đầu vào\n",
    "def as_coords(point):\n",
```

```
    "    \"\"\"Vector tọa độ (float, 1-D) từ MỘT điểm, dù đó là dãy
số/ndarray hay một đối tượng giống DataPoint (có thuộc tính .coords,\n",
    "    như docs/oop_design.md: DataPoint.coords). Lớp IncrementalDBSCAN
thật sẽ nhận DataPoint, còn code đo đạc của D05 đến D10 có thể\n",
    "    đưa tọa độ trần; RefitDBSCAN chấp nhận cả hai để đổi engine
không phải đổi nơi gọi.\"\"\"\n",
    "    raw = point.coords if hasattr(point, \"coords\") else point\n",
    "    return np.asarray(raw, dtype=float).ravel()\n",
    "\n",
    "\n",
    "def as_matrix(data):\n",
    "    \"\"\"Ma trận (n, d) từ ndarray/list 2-D, hoặc từ một dãy đối
tượng giống DataPoint (mỗi phần tử có .coords).\"\"\"\n",
    "    if isinstance(data, (list, tuple)) and len(data) and
hasattr(data[0], \"coords\"):\n",
    "        return np.vstack([as_coords(p) for p in data])\n",
    "    return np.asarray(data, dtype=float)\n",
    "\n",
    "\n",
    "class IdMap:\n",
    "    \"\"\"Ánh xạ rõ ràng giữa id trong stream (cột point_id của
d0.csv và updates.csv) và id do engine cấp.\n",
    "\n",
    "    Hai id khác nhau về nguyên tắc: engine cấp id riêng
(RefitDBSCAN: 0..n-1 cho fit, rồi tăng dần cho insert; engine thật của
nhóm có\n",
    "    thể cấp khác), còn nhãn xuất ra (labels_long.csv) luôn dùng id
của STREAM để các thuật toán so được với nhau.\n",
    "    \"\"\"\n",
    "\n",
    "    def __init__(self):\n",
    "        self._to_engine = {}\n",
    "        self._to_stream = {}\n",
    "\n",
    "    def bind(self, stream_id, engine_id):\n",
    "        stream_id, engine_id = int(stream_id), int(engine_id)\n",
    "        if stream_id in self._to_engine or engine_id in
self._to_stream:\n",
    "            raise KeyError(f\"id đã gắn: stream {stream_id} hoặc
engine {engine_id}\")\n",
    "        self._to_engine[stream_id], self._to_stream[engine_id] =
engine_id, stream_id\n",
    "\n",
    "    def engine_id(self, stream_id):\n",
    "        return self._to_engine[int(stream_id)]\n",
    "\n",
    "    def stream_id(self, engine_id):\n",
    "        return self._to_stream[int(engine_id)]\n",
    "\n",
    "    def release(self, stream_id):\n",
    "        \"\"\"Gỡ cặp id khi điểm bị xóa; trả id engine để gọi
engine.delete.\"\"\"\n",
    "        engine_id = self._to_engine.pop(int(stream_id))\n",
    "        del self._to_stream[engine_id]\n",
```

```
    "        return engine_id\n",
    "\n",
    "    def __len__(self):\n",
    "        return len(self._to_engine)\n",
    "\n",
    "\n",
    "#lớp thay thế tạm\n",
    "class RefitDBSCAN:\n",
    "    \"\"\"Baseline đối chứng: dùng sklearn DBSCAN và fit lại toàn bộ
sau mỗi insert/delete.\"\"\"\n",
    "\n",
    "    def __init__(self, eps, min_pts):\n",
    "        self.eps = float(eps)\n",
    "        self.min_pts = int(min_pts)\n",
    "        self._ids = []\n",
    "        self._coords = np.empty((0, 0))\n",
    "        self._labels = np.empty(0, dtype=int)\n",
    "        self._types = np.empty(0, dtype=int)\n",
    "        self._next_id = 0\n",
    "        self.last_update_seconds = 0.0\n",
    "\n",
    "    def _refit(self):\n",
    "        t0 = time.perf_counter()\n",
    "        self._labels, self._types = fit_full_dbscan(self._coords,
self.eps, self.min_pts)\n",
    "        self.last_update_seconds = time.perf_counter() - t0\n",
    "\n",
    "    def fit(self, data):\n",
    "        data = as_matrix(data)\n",
    "        if data.ndim != 2:\n",
    "            raise ValueError(\"fit cần ma trận 2-D (n_điểm,
n_chiều)\")\n",
    "        self._coords = data.copy()\n",
    "        self._ids = list(range(len(data)))\n",
    "        self._next_id = len(data)\n",
    "        self._refit()\n",
    "        return self\n",
    "\n",
    "    def insert(self, coords):\n",
    "        \"\"\"Thêm một điểm, fit lại toàn bộ, trả id của điểm mới.
`coords` là tọa độ trần hoặc đối tượng có .coords (DataPoint).\"\"\"\n",
    "        p = as_coords(coords)\n",
    "        if self._coords.size and p.shape[0] !=
self._coords.shape[1]:\n",
    "            raise ValueError(f\"điểm mới cần {self._coords.shape[1]}
chiều, nhận được {p.shape[0]}\")\n",
    "        self._coords = p[None, :] if self._coords.size == 0 else
np.vstack([self._coords, p])\n",
    "        pid = self._next_id\n",
    "        self._next_id += 1\n",
    "        self._ids.append(pid)\n",
    "        self._refit()\n",
    "        return pid\n",
    "\n",
```

```
    "    def delete(self, point_id):\n",
    "        \"\"\"Xóa điểm theo id, fit lại toàn bộ. KeyError nếu id
không tồn tại.\"\"\"\n",
    "        try:\n",
    "            pos = self._ids.index(point_id)\n",
    "        except ValueError:\n",
    "            raise KeyError(f\"không có điểm id={point_id}\") from
None\n",
    "        self._ids.pop(pos)\n",
    "        self._coords = np.delete(self._coords, pos, axis=0)\n",
    "        self._refit()\n",
    "\n",
    "    # Baseline dùng trực tiếp giao diện fit/insert/delete, không cần
bí danh riêng.\n",
    "    def get_labels(self):\n",
    "        \"\"\"Nhãn theo thứ tự id hiện tại (list int; -1 là
nhiễu).\"\"\"\n",
    "        return self._labels.tolist()\n",
    "\n",
    "    def snapshot(self):\n",
    "        \"\"\"Ảnh chụp trạng thái: dict gồm ids, coords, labels,
point_types (mảng numpy, bản sao).\"\"\"\n",
    "        return {\"ids\": np.array(self._ids, dtype=int), \"coords\":
self._coords.copy(),\n",
    "                \"labels\": self._labels.copy(), \"point_types\":
self._types.copy()}\n",
    "\n",
    "    @property\n",
    "    def n_points(self):\n",
    "        return len(self._ids)\n",
    "\n",
    "    @property\n",
    "    def n_clusters(self):\n",
    "        return len(set(self._labels.tolist()) - {-1})\n",
    "\n",
    "    @property\n",
    "    def n_noise(self):\n",
    "        return int((self._labels == -1).sum())\n",
    "\n",
    "\n",
    "#so sánh phân hoạch\n",
    "def _align(labels_a, labels_b, ids):\n",
    "    if isinstance(labels_a, Mapping) or isinstance(labels_b,
Mapping):\n",
    "        if not (isinstance(labels_a, Mapping) and
isinstance(labels_b, Mapping)):\n",
    "            raise ValueError(\"hai nhãn phải cùng là dict hoặc cùng
là dãy\")\n",
    "        if set(labels_a) != set(labels_b):\n",
    "            raise ValueError(f\"hai dict nhãn khác tập id:
{sorted(set(labels_a) ^ set(labels_b), key=str)}\")\n",
    "        ids = list(labels_a) if ids is None else list(ids)\n",
    "        return ids, np.array([labels_a[i] for i in ids]),
np.array([labels_b[i] for i in ids])\n",
```

```
    "    a, b = np.asarray(labels_a), np.asarray(labels_b)\n",
    "    if a.shape != b.shape:\n",
    "        raise ValueError(f\"độ dài khác nhau: {a.shape} và
{b.shape}\")\n",
    "    ids = list(range(len(a))) if ids is None else list(ids)\n",
    "    if len(ids) != len(a):\n",
    "        raise ValueError(\"ids phải cùng độ dài với nhãn\")\n",
    "    return ids, a, b\n",
    "\n",
    "\n",
    "def compare_partitions(labels_a, labels_b, ids=None,
ignore_ids=None, types_a=None, types_b=None):\n",
    "    \"\"\"So hai phân hoạch (id cụm không cần giống nhau, chỉ so
cách chia).\n",
    "\n",
    "    labels_a, labels_b: dãy cùng độ dài hoặc hai dict {id: nhãn}; -1
là nhiễu.\n",
    "    ids: id của từng vị trí (mặc định 0..n-1); ignore_ids: id bị bỏ
trước khi so (cột\n",
    "    ambiguous_border_ids của updates.csv, điểm biên mà hai cài đặt
đúng có thể gán khác cụm).\n",
    "    types_a, types_b (tùy chọn): mảng PointType, để so thêm tập
lõi.\n",
    "\n",
    "    Trả dict: n_compared, n_ignored, ari (nhiễu tính là một lớp),
noise_equal, noise_only_a,\n",
    "    noise_only_b, core_equal (None nếu thiếu types), partition_equal
(ari = 1 và tập nhiễu giống nhau).\n",
    "    \"\"\"\n",
    "    ids, a, b = _align(labels_a, labels_b, ids)\n",
    "    ignore = set(ignore_ids or [])\n",
    "    keep = np.array([i not in ignore for i in ids], dtype=bool)\n",
    "    ids_k = [i for i, k in zip(ids, keep) if k]\n",
    "    a, b = a[keep], b[keep]\n",
    "    na, nb = {i for i, v in zip(ids_k, a) if v == -1}, {i for i, v
in zip(ids_k, b) if v == -1}\n",
    "    ari = 1.0 if len(a) == 0 else float(adjusted_rand_score(a,
b))\n",
    "    core_equal = None\n",
    "    if types_a is not None and types_b is not None:\n",
    "        ta, tb = np.asarray(types_a)[keep],
np.asarray(types_b)[keep]\n",
    "        core_equal = {i for i, v in zip(ids_k, ta) if v ==
PointType.CORE} == \\\n",
    "                     {i for i, v in zip(ids_k, tb) if v ==
PointType.CORE}\n",
    "    return {\"n_compared\": int(len(a)), \"n_ignored\": int(len(ids)
- len(a)), \"ari\": ari,\n",
    "            \"noise_equal\": na == nb, \"noise_only_a\": sorted(na -
nb, key=str),\n",
    "            \"noise_only_b\": sorted(nb - na, key=str),
\"core_equal\": core_equal,\n",
    "            \"partition_equal\": bool(np.isclose(ari, 1.0) and na ==
nb)}\n",
```

```
    "\n",
    "\n",
    "#chạy lại stream\n",
    "def read_stream(stream_dir):\n",
    "    \"\"\"Đọc d0.csv, updates.csv, params.json; trả (d0, updates,
params, feature_cols).\"\"\"\n",
    "    stream_dir = Path(stream_dir)\n",
    "    d0 = pd.read_csv(stream_dir / \"d0.csv\", encoding=\"utf-
8\")\n",
    "    upd = pd.read_csv(stream_dir / \"updates.csv\",
dtype={\"ambiguous_border_ids\": str}, encoding=\"utf-8\")\n",
    "    params = json.loads((stream_dir /
\"params.json\").read_text(encoding=\"utf-8\"))\n",
    "    return d0, upd, params, feature_columns(d0, params)\n",
    "\n",
    "\n",
    "def ambiguous_by_step(params, upd):\n",
    "    \"\"\"Điểm biên mơ hồ theo từng step, dạng {step: [point_id]}.
Step 0 lấy từ params.json, các step sau từ cột\n",
    "    ambiguous_border_ids của updates.csv.\"\"\"\n",
    "    amb = {0: [int(v) for v in
params.get(\"d0_ambiguous_border_ids\", [])]}\n",
    "    for step, ids in zip(upd[\"step\"],
upd[\"ambiguous_border_ids\"]):\n",
    "        amb[int(step)] = [int(v) for v in str(ids).split(\";\") if v
and v != \"nan\"]\n",
    "    return amb\n"
   ]
  },
  {
   "cell_type": "markdown",
   "id": "19f6bf96",
   "metadata": {},
   "source": [
    "#### dbscan.py"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": null,
   "id": "257650b0",
   "metadata": {
    "vscode": {
     "languageId": "plaintext"
    }
   },
   "outputs": [],
   "source": [
    "from typing import Dict, List, Optional, Set\n",
    "\n",
    "import numpy as np\n",
    "\n",
    "from src.core.data_point import DataPoint, PointType\n",
    "from src.core.distance import DistanceMetric\n",
```

```
    "\n",
    "\n",
    "class DBSCAN:\n",
    "    \"\"\"DBSCAN tự cài theo giả mã dùng stack trong paper
IncrementalDBSCAN 1998.\n",
    "\n",
    "    Dùng stack LIFO để mở rộng cụm qua các điểm lõi; cache lân cận
được giữ\n",
    "    để lớp IncrementalDBSCAN dùng.\n",
    "\n",
    "    Attributes:\n",
    "        points: Các điểm, được tra cứu theo ID ổn định.\n",
    "        eps: Bán kính lân cận.\n",
    "        min_pts: Số điểm tối thiểu trong lân cận để một điểm là
core.\n",
    "    \"\"\"\n",
    "\n",
    "    def __init__(\n",
    "        self,\n",
    "        eps: float,\n",
    "        min_pts: int,\n",
    "        metric: Optional[DistanceMetric] = None,\n",
    "    ) -> None:\n",
    "        \"\"\"Tạo mô hình DBSCAN.\n",
    "\n",
    "        Args:\n",
    "            eps: Bán kính dùng để xác định epsilon-lân cận.\n",
    "            min_pts: Số điểm tối thiểu để xác định điểm core.\n",
    "            metric: Bộ tính khoảng cách; mặc định dùng
'DistanceMetric'.\n",
    "        \"\"\"\n",
    "        self.eps: float = eps\n",
    "        self.min_pts: int = min_pts\n",
    "        self.metric: DistanceMetric = metric if metric is not None
else DistanceMetric()\n",
    "        self.points: Dict[int, DataPoint] = {}\n",
    "        self._next_point_id: int = 0\n",
    "        self._next_cluster_id: int = 0\n",
    "\n",
    "    def _region_query(self, point_id: int) -> Set[int]:\n",
    "        \"\"\"Tìm các ID trong epsilon-lân cận của điểm.\n",
    "\n",
    "        Args:\n",
    "            point_id: ID điểm trung tâm của vùng truy vấn.\n",
    "\n",
    "        Returns:\n",
    "            Tập ID của các điểm cách điểm trung tâm không quá eps;
bao gồm\n",
    "            chính điểm trung tâm.\n",
    "\n",
    "        Raises:\n",
    "            ValueError: Nếu ID không tồn tại.\n",
    "        \"\"\"\n",
    "        try:\n",
```

```
    "            target_point = self.points[point_id]\n",
    "        except KeyError as error:\n",
    "            raise ValueError(f\"Không tìm thấy DataPoint có ID =
{point_id}\") from error\n",
    "\n",
    "        return {\n",
    "            candidate_id\n",
    "            for candidate_id, candidate in self.points.items()\n",
    "            if self.metric.euclidean(target_point.coords,
candidate.coords) <= self.eps\n",
    "        }\n",
    "\n",
    "    def fit(self, data: np.ndarray) -> \"DBSCAN\":\n",
    "        \"\"\"Phân cụm dữ liệu ban đầu và khởi tạo trạng thái lân
cận.\n",
    "\n",
    "        Args:\n",
    "            data: Ma trận có dạng '(số điểm, số chiều)'.\n",
    "\n",
    "        Returns:\n",
    "            Chính mô hình đã được huấn luyện.\n",
    "        \"\"\"\n",
    "        self.points = {i: DataPoint(i, row) for i, row in
enumerate(data)}\n",
    "        self._next_point_id = len(self.points)\n",
    "        self._next_cluster_id = 0\n",
    "\n",
    "        for point_id, point in self.points.items():\n",
    "            point.set_neighborhood(self._region_query(point_id))\n",
    "\n",
    "        for point_id, point in self.points.items():\n",
    "            if point.cluster_id is not None:\n",
    "                continue\n",
    "\n",
    "            if point.neighbor_count < self.min_pts:\n",
    "                point.cluster_id = -1\n",
    "                point.point_type = PointType.NOISE\n",
    "            else:\n",
    "                point.point_type = PointType.CORE\n",
    "                self._expand_cluster(\n",
    "                    point_id,\n",
    "                    point.neighbor_ids,\n",
    "                    self._next_cluster_id,\n",
    "                )\n",
    "                self._next_cluster_id += 1\n",
    "\n",
    "        return self\n",
    "\n",
    "    def _expand_cluster(self, seed_id: int, neighbor_ids: Set[int],
cluster_id: int) -> None:\n",
    "        \"\"\"Mở rộng cụm bằng cách duyệt các core và border tiếp
cận được.\n",
    "\n",
    "        Args:\n",
```

```
    "            seed_id: ID core khởi tạo cụm.\n",
    "            neighbor_ids: Lân cận của seed.\n",
    "            cluster_id: Nhãn cụm cần gán.\n",
    "        \"\"\"\n",
    "        seed = self.points[seed_id]\n",
    "        seed.cluster_id = cluster_id\n",
    "        seed.point_type = PointType.CORE\n",
    "        stack = sorted(neighbor_ids, reverse=True)\n",
    "\n",
    "        while stack:\n",
    "            current_id = stack.pop()\n",
    "            current_point = self.points[current_id]\n",
    "\n",
    "            if current_point.cluster_id == -1:\n",
    "                current_point.cluster_id = cluster_id\n",
    "                current_point.point_type = PointType.BORDER\n",
    "\n",
    "            if current_point.cluster_id is not None:\n",
    "                continue\n",
    "\n",
    "            current_point.cluster_id = cluster_id\n",
    "\n",
    "            if current_point.neighbor_count >= self.min_pts:\n",
    "                current_point.point_type = PointType.CORE\n",
    "                stack.extend(sorted(current_point.neighbor_ids,
reverse=True))\n",
    "            else:\n",
    "                current_point.point_type = PointType.BORDER\n",
    "\n",
    "    def get_labels(self) -> List[int]:\n",
    "        \"\"\"Trả nhãn cụm theo thứ tự điểm hiện lưu trong mô
hình.\n",
    "\n",
    "        Returns:\n",
    "            Danh sách nhãn; -1 biểu thị noise hoặc điểm chưa được
phân cụm.\n",
    "        \"\"\"\n",
    "        return [\n",
    "            point.cluster_id if point.cluster_id is not None else -
1\n",
    "            for point in self.points.values()\n",
    "        ]\n"
   ]
  },
  {
   "cell_type": "markdown",
   "id": "a76a8475",
   "metadata": {},
   "source": [
    "#### incremental_dbscan.py"
   ]
  },
  {
   "cell_type": "code",
```

```
   "execution_count": null,
   "id": "6708aa24",
   "metadata": {
    "vscode": {
     "languageId": "plaintext"
    }
   },
   "outputs": [],
   "source": [
    "\"\"\"Cập nhật cụm DBSCAN cục bộ sau mỗi lần chèn hoặc xóa
điểm.\"\"\"\n",
    "\n",
    "from collections import deque\n",
    "from typing import Dict, Iterable, List, Optional, Set, Union\n",
    "\n",
    "import numpy as np\n",
    "\n",
    "from src.clustering.dbscan import DBSCAN\n",
    "from src.clustering.utils import ClusterUtils\n",
    "from src.core.data_point import DataPoint, PointType\n",
    "\n",
    "\n",
    "class IncrementalDBSCAN(DBSCAN):\n",
    "    \"\"\"Cập nhật DBSCAN theo thuật toán IncrementalDBSCAN của
Ester et al. (1998).\n",
    "\n",
    "    Insert mở rộng từ UpdSeedIns bằng stack; delete xử lý Removal,
Reduction\n",
    "    và Potential Split bằng BFS bắt đầu từ UpdSeedDel.\n",
    "    \"\"\"\n",
    "\n",
    "    def insert(self, point: Union[DataPoint, Iterable[float]]) ->
int:\n",
    "        \"\"\"Chèn một điểm và cập nhật các điểm nằm trong vùng ảnh
hưởng.\n",
    "\n",
    "        Args:\n",
    "            point: 'DataPoint hoặc vector tọa độ của điểm mới.\n",
    "\n",
    "        Returns:\n",
    "            ID ổn định được cấp cho điểm mới.\n",
    "\n",
    "        Raises:\n",
    "            ValueError: Nếu tọa độ rỗng, không hữu hạn hoặc sai số
chiều.\n",
    "        \"\"\"\n",
    "        coords = self._coerce_coordinates(point)\n",
    "        new_point_id = self._next_point_id\n",
    "        self.points[new_point_id] = DataPoint(new_point_id,
coords)\n",
    "        self._update_neighborhood_after_insert(new_point_id)\n",
    "        self._next_point_id += 1\n",
    "\n",
    "        new_core_ids = self._find_new_core_points(new_point_id)\n",
```

```
    "        update_seed_ids = self._get_update_seeds(new_core_ids)\n",
    "        if update_seed_ids:\n",
    "            self._expand_new_connections(new_core_ids,
update_seed_ids)\n",
    "        else:\n",
    "            self._assign_border_or_noise(new_point_id)\n",
    "\n",
    "        return new_point_id\n",
    "\n",
    "    def _coerce_coordinates(self, point: Union[DataPoint,
Iterable[float]]) -> np.ndarray:\n",
    "        \"\"\"Chuẩn hóa và kiểm tra tọa độ trước khi thêm điểm.\n",
    "\n",
    "        Args:\n",
    "            point: Điểm dữ liệu hoặc vector tọa độ.\n",
    "\n",
    "        Returns:\n",
    "            Vector tọa độ một chiều kiểu số thực.\n",
    "\n",
    "        Raises:\n",
    "            ValueError: Nếu vector rỗng, chứa giá trị không hữu hạn
hoặc\n",
    "                không cùng số chiều với dữ liệu hiện tại.\n",
    "        \"\"\"\n",
    "        raw_coords = point.coords if isinstance(point, DataPoint)
else point\n",
    "        coords = np.asarray(raw_coords, dtype=float).reshape(-1)\n",
    "        if coords.size == 0 or not np.all(np.isfinite(coords)):\n",
    "            raise ValueError(\"Tọa độ phải khác rỗng và chứa các giá
trị hữu hạn.\")\n",
    "\n",
    "        if self.points:\n",
    "            expected_dimension =
next(iter(self.points.values())).dimension\n",
    "            if coords.size != expected_dimension:\n",
    "                raise ValueError(\n",
    "                    f\"Điểm mới phải có {expected_dimension} chiều,
\"\n",
    "                    f\"nhưng đang có {coords.size} chiều.\"\n",
    "                )\n",
    "        return coords\n",
    "\n",
    "    def insert_point(self, point: Union[DataPoint, Iterable[float]])
-> int:\n",
    "        \"\"\"Tên tương thích cũ; chuyển tiếp tới
:meth:'insert'.\n",
    "\n",
    "        Args:\n",
    "            point: 'DataPoint' hoặc vector tọa độ của điểm mới.\n",
    "\n",
    "        Returns:\n",
    "            ID ổn định được cấp cho điểm mới.\n",
    "        \"\"\"\n",
    "        return self.insert(point)\n",
```

```
    "\n",
    "    def _update_neighborhood_after_insert(self, new_point_id: int) -
> None:\n",
    "        \"\"\"Cập nhật cache lân cận hai chiều sau khi thêm
điểm.\n",
    "\n",
    "        Args:\n",
    "            new_point_id: ID của điểm vừa được thêm vào
'points'.\n",
    "\n",
    "        \"\"\"\n",
    "        neighbor_ids = self._region_query(new_point_id)\n",
    "        self.points[new_point_id].set_neighborhood(neighbor_ids)\n",
    "\n",
    "        for neighbor_id in sorted(neighbor_ids -
{new_point_id}):\n",
    "
self.points[neighbor_id].neighbor_ids.add(new_point_id)\n",
    "\n",
    "    def _find_new_core_points(self, new_point_id: int) ->
Set[int]:\n",
    "        \"\"\"Tìm điểm mới và điểm cũ vừa đạt ngưỡng 'min_pts'.\n",
    "\n",
    "        Args:\n",
    "            new_point_id: ID của điểm vừa được chèn.\n",
    "\n",
    "        Returns:\n",
    "            ID các điểm mới trở thành core.\n",
    "        \"\"\"\n",
    "        new_core_ids: Set[int] = set()\n",
    "        new_point = self.points[new_point_id]\n",
    "        if new_point.neighbor_count >= self.min_pts:\n",
    "            new_core_ids.add(new_point_id)\n",
    "\n",
    "        for neighbor_id in sorted(new_point.neighbor_ids):\n",
    "            neighbor = self.points[neighbor_id]\n",
    "            if neighbor.neighbor_count == self.min_pts:\n",
    "                new_core_ids.add(neighbor_id)\n",
    "        return new_core_ids\n",
    "\n",
    "    def _get_update_seeds(self, new_core_ids: Set[int]) ->
Set[int]:\n",
    "        \"\"\"Tạo tập 'UpdSeedIns' từ các core vừa phát sinh.\n",
    "\n",
    "        Args:\n",
    "            new_core_ids: ID các điểm vừa trở thành core.\n",
    "\n",
    "        Returns:\n",
    "            ID các điểm core trong lân cận của core mới.\n",
    "        \"\"\"\n",
    "        update_seed_ids: Set[int] = set()\n",
    "        for core_id in sorted(new_core_ids):\n",
    "            for neighbor_id in
sorted(self.points[core_id].neighbor_ids):\n",
```

```
    "                neighbor = self.points[neighbor_id]\n",
    "                if neighbor.neighbor_count >= self.min_pts:\n",
    "                    update_seed_ids.add(neighbor_id)\n",
    "        return update_seed_ids\n",
    "\n",
    "    def _assign_border_or_noise(self, new_point_id: int) ->
None:\n",
    "        \"\"\"Gán điểm mới thành border nếu kề core, nếu không thì
thành noise.\n",
    "\n",
    "        Args:\n",
    "            new_point_id: ID điểm mới không phát sinh cập nhật
seed.\n",
    "        \"\"\"\n",
    "        new_point = self.points[new_point_id]\n",
    "        core_cluster_ids: Set[int] = set()\n",
    "        for neighbor_id in new_point.neighbor_ids -
{new_point_id}:\n",
    "            neighbor = self.points[neighbor_id]\n",
    "            if (\n",
    "                neighbor.point_type == PointType.CORE\n",
    "                and neighbor.cluster_id is not None\n",
    "                and neighbor.cluster_id >= 0\n",
    "            ):\n",
    "                core_cluster_ids.add(neighbor.cluster_id)\n",
    "        if core_cluster_ids:\n",
    "            new_point.cluster_id = min(core_cluster_ids)\n",
    "            new_point.point_type = PointType.BORDER\n",
    "        else:\n",
    "            new_point.cluster_id = -1\n",
    "            new_point.point_type = PointType.NOISE\n",
    "\n",
    "    def _expand_new_connections(\n",
    "        self,\n",
    "        new_core_ids: Set[int],\n",
    "        update_seed_ids: Set[int],\n",
    "    ) -> None:\n",
    "        \"\"\"Mở rộng DBSCAN trong vùng bị ảnh hưởng, bắt đầu từ
UpdSeedIns.\"\"\"\n",
    "        components = self._get_insert_core_components(new_core_ids,
update_seed_ids)\n",
    "        parents = list(range(len(components)))\n",
    "\n",
    "        def find(component_id: int) -> int:\n",
    "            while parents[component_id] != component_id:\n",
    "                parents[component_id] =
parents[parents[component_id]]\n",
    "                component_id = parents[component_id]\n",
    "            return component_id\n",
    "\n",
    "        def union(first_id: int, second_id: int) -> None:\n",
    "            first_root = find(first_id)\n",
    "            second_root = find(second_id)\n",
    "            if first_root != second_root:\n",
```

```
    "                parents[max(first_root, second_root)] =
min(first_root, second_root)\n",
    "\n",
    "        old_clusters_by_component: Dict[int, Set[int]] = {}\n",
    "        component_by_old_cluster: Dict[int, int] = {}\n",
    "        for component_id, component in enumerate(components):\n",
    "            old_cluster_ids: Set[int] = set()\n",
    "            for core_id in component:\n",
    "                previous_cluster_id =
self.points[core_id].cluster_id\n",
    "                if previous_cluster_id is not None and
previous_cluster_id >= 0:\n",
    "                    old_cluster_ids.add(previous_cluster_id)\n",
    "\n",
    "                old_core_seed_ids = (\n",
    "                    self.points[core_id].neighbor_ids &
update_seed_ids\n",
    "                ) - new_core_ids\n",
    "                for seed_id in old_core_seed_ids:\n",
    "                    old_seed = self.points[seed_id]\n",
    "                    if (\n",
    "                        old_seed.is_core\n",
    "                        and old_seed.cluster_id is not None\n",
    "                        and old_seed.cluster_id >= 0\n",
    "                    ):\n",
    "                        old_cluster_ids.add(old_seed.cluster_id)\n",
    "\n",
    "            old_clusters_by_component[component_id] =
old_cluster_ids\n",
    "            for cluster_id in sorted(old_cluster_ids):\n",
    "                previous_component =
component_by_old_cluster.setdefault(\n",
    "                    cluster_id, component_id\n",
    "                )\n",
    "                # Các nhóm lõi mới cùng nối vào một cụm cũ thuộc
cùng một cụm sau cập nhật.\n",
    "                union(component_id, previous_component)\n",
    "\n",
    "        core_ids_by_group: Dict[int, Set[int]] = {}\n",
    "        old_cluster_ids_by_group: Dict[int, Set[int]] = {}\n",
    "        for component_id, component in enumerate(components):\n",
    "            root_id = find(component_id)\n",
    "            core_ids_by_group.setdefault(root_id,
set()).update(component)\n",
    "            old_cluster_ids_by_group.setdefault(root_id,
set()).update(\n",
    "                old_clusters_by_component[component_id]\n",
    "            )\n",
    "\n",
    "        for group_id in sorted(core_ids_by_group):\n",
    "            old_cluster_ids = old_cluster_ids_by_group[group_id]\n",
    "            if old_cluster_ids:\n",
    "                target_cluster_id = min(old_cluster_ids)\n",
```

```
    "                # Absorption giữ một cụm cũ; Merge hợp nhất các cụm
cũ được nối.\n",
    "                for source_cluster_id in sorted(old_cluster_ids -
{target_cluster_id}):\n",
    "                    ClusterUtils.merge_clusters(\n",
    "                        self.points, source_cluster_id,
target_cluster_id\n",
    "                    )\n",
    "            else:\n",
    "                # Creation: nhóm lõi mới chưa nối với cụm nào trước
đó.\n",
    "                target_cluster_id = self._next_cluster_id\n",
    "                self._next_cluster_id += 1\n",
    "\n",
    "            for core_id in core_ids_by_group[group_id]:\n",
    "                self.points[core_id].cluster_id =
target_cluster_id\n",
    "                self.points[core_id].point_type = PointType.CORE\n",
    "\n",
    "        self._relabel_borders_near_new_cores(new_core_ids)\n",
    "\n",
    "    def _get_insert_core_components(\n",
    "        self,\n",
    "        new_core_ids: Set[int],\n",
    "        update_seed_ids: Set[int],\n",
    "    ) -> List[Set[int]]:\n",
    "        \"\"\"Duyệt các điểm NewCore trong UpdSeedIns bằng stack
LIFO.\"\"\"\n",
    "        remaining_ids = set(new_core_ids) & set(update_seed_ids)\n",
    "        components: List[Set[int]] = []\n",
    "        while remaining_ids:\n",
    "            start_id = min(remaining_ids)\n",
    "            remaining_ids.remove(start_id)\n",
    "            stack = [start_id]\n",
    "            component: Set[int] = set()\n",
    "            while stack:\n",
    "                current_id = stack.pop()\n",
    "                component.add(current_id)\n",
    "                for neighbor_id in sorted(\n",
    "                    self.points[current_id].neighbor_ids &
remaining_ids,\n",
    "                    reverse=True,\n",
    "                ):\n",
    "                    remaining_ids.remove(neighbor_id)\n",
    "                    stack.append(neighbor_id)\n",
    "            components.append(component)\n",
    "        return components\n",
    "\n",
    "    def _relabel_borders_near_new_cores(self, new_core_ids:
Set[int]) -> None:\n",
    "        \"\"\"Gán nhãn cho border mới và noise được core mới hấp
thụ.\n",
    "\n",
    "        Args:\n",
```

```
    "            new_core_ids: ID các core vừa phát sinh.\n",
    "\n",
    "        Raises:\n",
    "            RuntimeError: Nếu một điểm được xác định là border nhưng
không kề core.\n",
    "        \"\"\"\n",
    "        affected_border_ids = {\n",
    "            neighbor_id\n",
    "            for core_id in new_core_ids\n",
    "            for neighbor_id in self.points[core_id].neighbor_ids\n",
    "            if self.points[neighbor_id].neighbor_count <
self.min_pts\n",
    "        }\n",
    "        for border_id in sorted(affected_border_ids):\n",
    "            border = self.points[border_id]\n",
    "            if border.cluster_id is None or border.cluster_id == -
1:\n",
    "                adjacent_cluster_ids: Set[int] = set()\n",
    "                for neighbor_id in border.neighbor_ids:\n",
    "                    neighbor = self.points[neighbor_id]\n",
    "                    if (\n",
    "                        neighbor.point_type == PointType.CORE\n",
    "                        and neighbor.cluster_id is not None\n",
    "                        and neighbor.cluster_id >= 0\n",
    "                    ):\n",
    "
adjacent_cluster_ids.add(neighbor.cluster_id)\n",
    "                if not adjacent_cluster_ids:\n",
    "                    raise RuntimeError(\"Border không nằm trong lân
cận của Core nào.\")\n",
    "                border.cluster_id = min(adjacent_cluster_ids)\n",
    "            border.point_type = PointType.BORDER\n",
    "\n",
    "    def delete(self, point_id: int) -> None:\n",
    "        \"\"\"Xóa một điểm và cập nhật cụm theo tập
'UpdSeedDel'.\n",
    "\n",
    "        Args:\n",
    "            point_id: ID ổn định của điểm cần xóa.\n",
    "\n",
    "        Raises:\n",
    "            ValueError: Nếu ID không tồn tại trong mô hình.\n",
    "        \"\"\"\n",
    "        if point_id not in self.points:\n",
    "            raise ValueError(f\"Không tìm thấy DataPoint có ID =
{point_id}\")\n",
    "\n",
    "        deleted_point = self.points[point_id]\n",
    "        deleted_neighbor_ids = deleted_point.neighbor_ids -
{point_id}\n",
    "        lost_core_ids = {point_id} if deleted_point.is_core else
set()\n",
    "\n",
```

```
    "        # Một Core kề p mất tính Core khi số láng giềng trước khi
xóa đúng MinPts.\n",
    "        for neighbor_id in deleted_neighbor_ids:\n",
    "            neighbor = self.points[neighbor_id]\n",
    "            if neighbor.is_core and neighbor.neighbor_count ==
self.min_pts:\n",
    "                lost_core_ids.add(neighbor_id)\n",
    "\n",
    "        affected_cluster_ids = {\n",
    "            self.points[lost_id].cluster_id\n",
    "            for lost_id in lost_core_ids - {point_id}\n",
    "            if self.points[lost_id].cluster_id is not None\n",
    "            and self.points[lost_id].cluster_id >= 0\n",
    "        }\n",
    "        if (\n",
    "            deleted_point.is_core\n",
    "            and deleted_point.cluster_id is not None\n",
    "            and deleted_point.cluster_id >= 0\n",
    "        ):\n",
    "            affected_cluster_ids.add(deleted_point.cluster_id)\n",
    "\n",
    "        self.points.pop(point_id)\n",
    "        self._remove_from_neighbor_caches(point_id,
deleted_point.neighbor_ids)\n",
    "        if not lost_core_ids:\n",
    "            return\n",
    "\n",
    "        for lost_id in lost_core_ids - {point_id}:\n",
    "            self.points[lost_id].point_type = PointType.BORDER\n",
    "\n",
    "        update_seed_ids =
self._get_delete_update_seeds(deleted_point, lost_core_ids)\n",
    "        seeds_by_cluster: Dict[int, Set[int]] = {}\n",
    "        for seed_id in update_seed_ids:\n",
    "            cluster_id = self.points[seed_id].cluster_id\n",
    "            if cluster_id is not None and cluster_id >= 0:\n",
    "                seeds_by_cluster.setdefault(cluster_id,
set()).add(seed_id)\n",
    "\n",
    "        # N_Eps(p) và N_Eps(q') là các điểm có thể đổi
Core/Border/Noise.\n",
    "        affected_point_ids = set(deleted_neighbor_ids)\n",
    "        for lost_id in lost_core_ids:\n",
    "            if lost_id == point_id:\n",
    "                affected_point_ids.update(deleted_neighbor_ids)\n",
    "            else:\n",
    "                affected_point_ids.add(lost_id)\n",
    "
affected_point_ids.update(self.points[lost_id].neighbor_ids)\n",
    "\n",
    "        for cluster_id in sorted(affected_cluster_ids):\n",
    "            seed_ids = seeds_by_cluster.get(cluster_id, set())\n",
    "            if not seed_ids:\n",
    "                # Removal: không còn Core nào trong UpdSeedDel.\n",
```

```
    "                continue\n",
    "\n",
    "            if self._are_seeds_directly_connected(seed_ids):\n",
    "                # Reduction: các seed trực tiếp nối nhau nên cụm
không thể tách.\n",
    "                continue\n",
    "\n",
    "            components = self._find_delete_components(seed_ids,
cluster_id)\n",
    "            if components is None:\n",
    "                # BFS tìm được đường nối qua các Core trung gian:
Non-split.\n",
    "                continue\n",
    "\n",
    "            # Potential Split: giữ ID cũ cho thành phần đầu tiên,
cấp ID mới\n",
    "            # cho các thành phần còn lại.\n",
    "            components.sort(key=min)\n",
    "            for index, component in enumerate(components):\n",
    "                new_cluster_id = cluster_id if index == 0 else
self._next_cluster_id\n",
    "                if index > 0:\n",
    "                    self._next_cluster_id += 1\n",
    "                for core_id in component:\n",
    "                    self.points[core_id].cluster_id =
new_cluster_id\n",
    "
affected_point_ids.update(self.points[core_id].neighbor_ids)\n",
    "\n",
    "        self._relabel_after_delete(affected_point_ids)\n",
    "\n",
    "    def _remove_from_neighbor_caches(\n",
    "        self,\n",
    "        deleted_point_id: int,\n",
    "        neighbor_ids: Set[int],\n",
    "    ) -> None:\n",
    "        \"\"\"Loại bỏ ID đã xóa khỏi cache lân cận của các điểm còn
lại.\n",
    "\n",
    "        Args:\n",
    "            deleted_point_id: ID điểm vừa bị xóa.\n",
    "            neighbor_ids: Lân cận được lưu trước khi xóa.\n",
    "        \"\"\"\n",
    "        for neighbor_id in neighbor_ids:\n",
    "            neighbor = self.points.get(neighbor_id)\n",
    "            if neighbor is not None:\n",
    "                neighbor.neighbor_ids.discard(deleted_point_id)\n",
    "\n",
    "    def _get_delete_update_seeds(\n",
    "        self,\n",
    "        deleted_point: DataPoint,\n",
    "        lost_core_ids: Set[int],\n",
    "    ) -> Set[int]:\n",
```

```
    "        \"\"\"Tính UpdSeedDel từ các Core vừa mất tính
Core.\"\"\"\n",
    "        update_seed_ids: Set[int] = set()\n",
    "        for lost_core_id in lost_core_ids:\n",
    "            neighbor_ids = (\n",
    "                deleted_point.neighbor_ids - {deleted_point.id}\n",
    "                if lost_core_id == deleted_point.id\n",
    "                else self.points[lost_core_id].neighbor_ids\n",
    "            )\n",
    "            update_seed_ids.update(\n",
    "                neighbor_id\n",
    "                for neighbor_id in neighbor_ids\n",
    "                if self.points[neighbor_id].is_core\n",
    "            )\n",
    "        return update_seed_ids\n",
    "\n",
    "    def _are_seeds_directly_connected(self, seed_ids: Set[int]) ->
bool:\n",
    "        \"\"\"Kiểm tra trường hợp Reduction: các seed đôi một kề
trực tiếp.\"\"\"\n",
    "        ordered_seeds = sorted(seed_ids)\n",
    "        return all(\n",
    "            second_id in self.points[first_id].neighbor_ids\n",
    "            for index, first_id in enumerate(ordered_seeds)\n",
    "            for second_id in ordered_seeds[index + 1 :]\n",
    "        )\n",
    "\n",
    "    def _find_delete_components(\n",
    "        self,\n",
    "        seed_ids: Set[int],\n",
    "        cluster_id: int,\n",
    "    ) -> Optional[List[Set[int]]]:\n",
    "        \"\"\"Chạy BFS song song từ UpdSeedDel để phân loại
Split/Non-split.\n",
    "\n",
    "        Trả None khi các seed đã nối được với nhau; nếu không, trả
các thành\n",
    "        phần lõi tìm được trong cụm cũ.\n",
    "        \"\"\"\n",
    "        parents = {seed_id: seed_id for seed_id in seed_ids}\n",
    "        owner = {seed_id: seed_id for seed_id in seed_ids}\n",
    "        queue = deque(sorted(seed_ids))\n",
    "\n",
    "        def find(seed_id: int) -> int:\n",
    "            while parents[seed_id] != seed_id:\n",
    "                parents[seed_id] = parents[parents[seed_id]]\n",
    "                seed_id = parents[seed_id]\n",
    "            return seed_id\n",
    "\n",
    "        def union(first_seed: int, second_seed: int) -> None:\n",
    "            first_root = find(first_seed)\n",
    "            second_root = find(second_seed)\n",
    "            if first_root != second_root:\n",
```

```
    "                parents[max(first_root, second_root)] =
min(first_root, second_root)\n",
    "\n",
    "        while queue:\n",
    "            current_id = queue.popleft()\n",
    "            current_seed = find(owner[current_id])\n",
    "            for neighbor_id in
sorted(self.points[current_id].neighbor_ids):\n",
    "                neighbor = self.points[neighbor_id]\n",
    "                if not neighbor.is_core or neighbor.cluster_id !=
cluster_id:\n",
    "                    continue\n",
    "\n",
    "                if neighbor_id not in owner:\n",
    "                    owner[neighbor_id] = current_seed\n",
    "                    queue.append(neighbor_id)\n",
    "                else:\n",
    "                    union(current_seed, owner[neighbor_id])\n",
    "                    if len({find(seed_id) for seed_id in seed_ids})
== 1:\n",
    "                        return None\n",
    "\n",
    "        components_by_seed: Dict[int, Set[int]] = {}\n",
    "        for point_id, seed_id in owner.items():\n",
    "            components_by_seed.setdefault(find(seed_id),
set()).add(point_id)\n",
    "        return list(components_by_seed.values())\n",
    "\n",
    "    def _relabel_after_delete(self, affected_point_ids: Set[int]) ->
None:\n",
    "        \"\"\"Đồng bộ border/noise và loại bỏ nhãn cụm không còn
core.\n",
    "\n",
    "        Args:\n",
    "            affected_point_ids: ID điểm cần gán lại trạng thái sau
khi tái cấu trúc.\n",
    "        \"\"\"\n",
    "        ClusterUtils.relabel_unreachable_points(\n",
    "            self.points, affected_point_ids, self._region_query\n",
    "        )\n",
    "        active_cluster_ids = {\n",
    "            point.cluster_id\n",
    "            for point in self.points.values()\n",
    "            if point.is_core and point.cluster_id is not None and
point.cluster_id >= 0\n",
    "        }\n",
    "        ClusterUtils.clean_stale_cluster_ids(self.points,
active_cluster_ids)\n",
    "\n",
    "    def delete_point(self, point_id: int) -> None:\n",
    "        \"\"\"Tên tương thích cũ; chuyển tiếp tới
:meth:'delete'.\n",
    "\n",
    "        Args:\n",
```

```
    "            point_id: ID ổn định của điểm cần xóa.\n",
    "\n",
    "        Raises:\n",
    "            ValueError: Nếu ID không tồn tại trong mô hình.\n",
    "        \"\"\"\n",
    "        self.delete(point_id)\n"
   ]
  },
  {
   "cell_type": "markdown",
   "id": "c7cb0c2e",
   "metadata": {},
   "source": [
    "#### utils.py"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": null,
   "id": "e8e3e81f",
   "metadata": {
    "vscode": {
     "languageId": "plaintext"
    }
   },
   "outputs": [],
   "source": [
    "from typing import Callable, Dict, Set\n",
    "\n",
    "from src.core.data_point import DataPoint, PointType\n",
    "\n",
    "\n",
    "class ClusterUtils:\n",
    "    \"\"\"Các hàm tiện ích xử lý hợp cụm, tách cụm và đồng bộ nhãn
trong Incremental DBSCAN.\"\"\"\n",
    "\n",
    "    @staticmethod\n",
    "    def merge_clusters(points: Dict[int, DataPoint], source_id: int,
target_id: int) -> None:\n",
    "        \"\"\"Đổi nhãn mọi điểm thuộc cụm nguồn sang cụm đích.\n",
    "\n",
    "        Args:\n",
    "            points: Các điểm được lập chỉ mục theo ID.\n",
    "            source_id: Nhãn cụm cần gộp.\n",
    "            target_id: Nhãn cụm sẽ được giữ lại.\n",
    "        \"\"\"\n",
    "        for point in points.values():\n",
    "            if point.cluster_id == source_id:\n",
    "                point.cluster_id = target_id\n",
    "\n",
    "    @staticmethod\n",
    "    def relabel_unreachable_points(\n",
    "        points: Dict[int, DataPoint],\n",
    "        affected_ids: Set[int],\n",
```

```
    "        region_query_fn: Callable[[int], Set[int]],\n",
    "    ) -> None:\n",
    "        \"\"\"Gán lại border/noise cho điểm không còn kề core sau
khi cập nhật.\n",
    "\n",
    "        Args:\n",
    "            points: Các điểm được lập chỉ mục theo ID.\n",
    "            affected_ids: Các điểm cần xác định lại nhãn.\n",
    "            region_query_fn: Hàm lấy lân cận theo ID điểm.\n",
    "        \"\"\"\n",
    "        for point_id in sorted(affected_ids):\n",
    "            point = points[point_id]\n",
    "            if point.is_core:\n",
    "                continue\n",
    "\n",
    "            neighbors = region_query_fn(point_id)\n",
    "            core_neighbors = [\n",
    "                neighbor_id\n",
    "                for neighbor_id in sorted(neighbors)\n",
    "                if points[neighbor_id].is_core\n",
    "                and points[neighbor_id].cluster_id is not None\n",
    "                and points[neighbor_id].cluster_id >= 0\n",
    "            ]\n",
    "\n",
    "            if core_neighbors:\n",
    "                point.cluster_id =
points[core_neighbors[0]].cluster_id\n",
    "                point.point_type = PointType.BORDER\n",
    "            else:\n",
    "                point.cluster_id = -1\n",
    "                point.point_type = PointType.NOISE\n",
    "\n",
    "    @staticmethod\n",
    "    def clean_stale_cluster_ids(points: Dict[int, DataPoint],
active_cluster_ids: Set[int]) -> None:\n",
    "        \"\"\"Đưa điểm về noise nếu nhãn cụm không còn core nào sở
hữu.\n",
    "\n",
    "        Args:\n",
    "            points: Các điểm được lập chỉ mục theo ID.\n",
    "            active_cluster_ids: Nhãn cụm còn ít nhất một điểm
core.\n",
    "        \"\"\"\n",
    "        for point in points.values():\n",
    "            if (\n",
    "                point.cluster_id is not None\n",
    "                and point.cluster_id >= 0\n",
    "                and point.cluster_id not in active_cluster_ids\n",
    "            ):\n",
    "                point.cluster_id = -1\n",
    "                point.point_type = PointType.NOISE\n"
  ]
 },
 {
```

```
   "cell_type": "markdown",
   "id": "8cf16bcf",
   "metadata": {},
   "source": [
    "#### data_point.py"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": null,
   "id": "5b4cce5d",
   "metadata": {
    "vscode": {
     "languageId": "plaintext"
    }
   },
   "outputs": [],
   "source": [
    "from enum import Enum\n",
    "from typing import Iterable, Optional, Set\n",
    "\n",
    "import numpy as np\n",
    "\n",
    "\n",
    "class PointType(Enum):\n",
    "    \"\"\"Trạng thái của một điểm theo DBSCAN.\"\"\"\n",
    "\n",
    "    UNCLASSIFIED = \"UNCLASSIFIED\"\n",
    "    CORE = \"CORE\"\n",
    "    BORDER = \"BORDER\"\n",
    "    NOISE = \"NOISE\"\n",
    "\n",
    "\n",
    "class DataPoint:\n",
    "    \"\"\"Biểu diễn một điểm dữ liệu trong không gian n chiều tổng
quát.\"\"\"\n",
    "\n",
    "    def __init__(self, point_id: int, coords: Iterable[float]) ->
None:\n",
    "        \"\"\"Khởi tạo điểm với ID ổn định và tọa độ n chiều.\n",
    "\n",
    "        Args:\n",
    "            point_id: ID duy nhất do mô hình quản lý.\n",
    "            coords: Các tọa độ của điểm.\n",
    "\n",
    "        Raises:\n",
    "            ValueError: Nếu không có tọa độ nào.\n",
    "        \"\"\"\n",
    "        self.id: int = point_id\n",
    "        self.coords: np.ndarray = np.array(coords, dtype=float,
copy=True).reshape(-1)\n",
    "        if self.coords.size == 0:\n",
    "            raise ValueError(\"Tọa độ của điểm phải có ít nhất một
chiều.\")\n",
```

```
    "\n",
    "        self.cluster_id: Optional[int] = None  # None: Chưa duyệt, -
1: Noise, >=0: Cluster ID\n",
    "        self.point_type: PointType = PointType.UNCLASSIFIED\n",
    "        self.neighbor_ids: Set[int] = set()\n",
    "\n",
    "    @property\n",
    "    def neighbor_count(self) -> int:\n",
    "        \"\"\"Số láng giềng, suy ra từ cache để tránh lệch bộ
đếm.\"\"\"\n",
    "        return len(self.neighbor_ids)\n",
    "\n",
    "    def set_neighborhood(self, neighbor_ids: Iterable[int]) ->
None:\n",
    "        \"\"\"Thay cache lân cận bằng các ID ổn định của điểm láng
giềng.\n",
    "\n",
    "        Args:\n",
    "            neighbor_ids: ID điểm nằm trong epsilon-lân cận.\n",
    "        \"\"\"\n",
    "        self.neighbor_ids = set(neighbor_ids)\n",
    "\n",
    "    @property\n",
    "    def is_core(self) -> bool:\n",
    "        \"\"\"Kiểm tra điểm có phải là điểm lõi (CORE Point) hay
không.\"\"\"\n",
    "        return self.point_type == PointType.CORE\n",
    "\n",
    "    @property\n",
    "    def dimension(self) -> int:\n",
    "        \"\"\"Trả về số chiều dữ liệu của điểm.\"\"\"\n",
    "        return self.coords.size\n",
    "\n",
    "    def __repr__(self) -> str:\n",
    "        \"\"\"Trả về biểu diễn ngắn gọn để kiểm tra trạng thái
điểm.\"\"\"\n",
    "        return (f\"DataPoint(id={self.id},
coords={self.coords.tolist()}, \"\n",
    "                f\"cluster={self.cluster_id},
type={self.point_type.value}, \"\n",
    "                f\"neighbor_ids={sorted(self.neighbor_ids)})\")\n"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": null,
   "id": "72a4a332",
   "metadata": {
    "vscode": {
     "languageId": "plaintext"
    }
   },
   "outputs": [],
   "source": [
```

```
    "\"\"\"Custom and reference metrics for evaluating clustering
results.\"\"\"\n",
    "\n",
    "from .metrics import silhouette_samples, silhouette_score\n",
    "\n",
    "__all__ = [\"silhouette_samples\", \"silhouette_score\"]\n"
   ]
  },
  {
   "cell_type": "markdown",
   "id": "201f191f",
   "metadata": {},
   "source": [
    "#### distance.py"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": null,
   "id": "2c970ebc",
   "metadata": {
    "vscode": {
     "languageId": "plaintext"
    }
   },
   "outputs": [],
   "source": [
    "from math import sqrt\n",
    "from typing import Iterable\n",
    "\n",
    "import numpy as np\n",
    "\n",
    "\n",
    "class DistanceMetric:\n",
    "    \"\"\"Tính khoảng cách Euclidean độc lập với lớp
DataPoint.\"\"\"\n",
    "\n",
    "    @staticmethod\n",
    "    def euclidean(first: Iterable[float], second: Iterable[float]) -
> float:\n",
    "        \"\"\"Tính khoảng cách Euclidean giữa hai vector cùng số
chiều.\n",
    "\n",
    "        Args:\n",
    "            first: Tọa độ vector thứ nhất.\n",
    "            second: Tọa độ vector thứ hai.\n",
    "\n",
    "        Returns:\n",
    "            Khoảng cách Euclidean.\n",
    "\n",
    "        Raises:\n",
    "            ValueError: Nếu hai vector có số chiều khác nhau.\n",
    "        \"\"\"\n",
```

```
    "        first_coords = np.asarray(first, dtype=float).reshape(-
1)\n",
    "        second_coords = np.asarray(second, dtype=float).reshape(-
1)\n",
    "        if first_coords.size != second_coords.size:\n",
    "            raise ValueError(\n",
    "                \"Hai vector phải có cùng số chiều: \"\n",
    "                f\"{first_coords.size}D và
{second_coords.size}D.\"\n",
    "            )\n",
    "\n",
    "        squared_distance = sum(\n",
    "            (float(first_value) - float(second_value)) ** 2\n",
    "            for first_value, second_value in zip(first_coords,
second_coords)\n",
    "        )\n",
    "        return sqrt(squared_distance)\n"
   ]
  },
  {
   "cell_type": "markdown",
   "id": "5318df48",
   "metadata": {},
   "source": [
    "#### equivalence.py"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": null,
   "id": "b6fc6d22",
   "metadata": {
    "vscode": {
     "languageId": "plaintext"
    }
   },
   "outputs": [],
   "source": [
    "\"\"\"Đối chiếu Incremental DBSCAN với DBSCAN chạy lại toàn bộ bằng
scikit-learn .\n",
    "\n",
    "Việc sử dụng thư viện chỉ giới hạn trong phần đánh giá và so
sánh.\n",
    "Thuật toán Incremental DBSCAN vẫn do nhóm tự cài đặt.\n",
    "\"\"\"\n",
    "\n",
    "from __future__ import annotations\n",
    "\n",
    "from pathlib import Path\n",
    "from typing import Any\n",
    "\n",
    "import pandas as pd\n",
    "\n",
    "from baseline.sklearn_dbscan import (\n",
```

```
    "    RefitDBSCAN,\n",
    "    ambiguous_by_step,\n",
    "    compare_partitions,\n",
    "    read_stream,\n",
    ")\n",
    "from src.clustering.incremental_dbscan import IncrementalDBSCAN\n",
    "from src.visualization.scenarios import (\n",
    "    list_scenarios,\n",
    "    load_scenario,\n",
    "    run_scenario,\n",
    "    stream_snapshots,\n",
    ")\n",
    "\n",
    "ROOT = Path(__file__).resolve().parents[2]\n",
    "RAW = ROOT / \"data\" / \"raw\"\n",
    "STREAMS = (\n",
    "    \"card_stream\",\n",
    ")\n",
    "\n",
    "\n",
    "def _compare_snapshots(\n",
    "    case: str,\n",
    "    step: int,\n",
    "    operation: str,\n",
    "    custom: dict[str, Any],\n",
    "    reference: dict[str, Any],\n",
    "    *,\n",
    "    ignore_ids: set[Any] | None = None,\n",
    "    eps: float,\n",
    "    min_pts: int,\n",
    ") -> dict[str, Any]:\n",
    "    \"\"\"So sánh hai trạng thái phân cụm theo ID điểm\"\"\"\n",
    "    custom_ids = [value.item() if hasattr(value, \"item\") else
value for value in custom[\"ids\"]]\n",
    "    reference_ids = [value.item() if hasattr(value, \"item\") else
value for value in reference[\"ids\"]]\n",
    "    custom_labels = dict(zip(custom_ids, map(int,
custom[\"labels\"]))))\n",
    "    reference_labels = dict(zip(reference_ids, map(int,
reference[\"labels\"]))))\n",
    "    if set(custom_labels) != set(reference_labels):\n",
    "        raise ValueError(f\"{case} step {step}: custom và sklearn có
tập điểm khác nhau\")\n",
    "\n",
    "    ids = sorted(custom_labels, key=str)\n",
    "    custom_types = dict(zip(custom_ids, map(int,
custom[\"point_types\"]))))\n",
    "    reference_types = dict(zip(reference_ids, map(int,
reference[\"point_types\"]))))\n",
    "    comparison = compare_partitions(\n",
    "        custom_labels,\n",
    "        reference_labels,\n",
    "        ids=ids,\n",
    "        ignore_ids=ignore_ids,\n",
```

```
    "        types_a=[custom_types[point_id] for point_id in ids],\n",
    "        types_b=[reference_types[point_id] for point_id in ids],\n",
    "    )\n",
    "    custom_values = list(custom_labels.values())\n",
    "    reference_values = list(reference_labels.values())\n",
    "    core_equal = comparison[\"core_equal\"]\n",
    "    equivalent = comparison[\"partition_equal\"] and core_equal is
True\n",
    "    return {\n",
    "        \"case\": case,\n",
    "        \"step\": step,\n",
    "        \"operation\": operation,\n",
    "        \"n_points\": len(ids),\n",
    "        \"n_compared\": comparison[\"n_compared\"],\n",
    "        \"n_ignored_ambiguous_border\":
comparison[\"n_ignored\"],\n",
    "        \"eps\": eps,\n",
    "        \"min_pts\": min_pts,\n",
    "        \"n_clusters_incremental\": len(set(custom_values) - {-
1}),\n",
    "        \"n_clusters_sklearn\": len(set(reference_values) - {-
1}),\n",
    "        \"n_noise_incremental\": sum(value == -1 for value in
custom_values),\n",
    "        \"n_noise_sklearn\": sum(value == -1 for value in
reference_values),\n",
    "        \"ari\": comparison[\"ari\"],\n",
    "        \"noise_equal\": comparison[\"noise_equal\"],\n",
    "        \"core_equal\": core_equal,\n",
    "        \"equivalent\": equivalent,\n",
    "    }\n",
    "\n",
    "\n",
    "def _scenario_rows(kind: str, name: str, raw_dir: Path = RAW) ->
list[dict[str, Any]]:\n",
    "    scenario = load_scenario(kind, name, raw_dir)\n",
    "    custom = run_scenario(IncrementalDBSCAN, scenario)\n",
    "    reference = run_scenario(RefitDBSCAN, scenario)\n",
    "    expected = scenario[\"expected\"]\n",
    "    global_ambiguous = set(expected.get(\"ambiguous_to_ignore\",
[]))\n",
    "    rows = [\n",
    "        _compare_snapshots(\n",
    "            f\"{kind}/{name}\",\n",
    "            0,\n",
    "            \"fit\",\n",
    "            custom[\"before\"],\n",
    "            reference[\"before\"],\n",
    "            ignore_ids=global_ambiguous,\n",
    "            eps=scenario[\"eps\"],\n",
    "            min_pts=scenario[\"min_pts\"],\n",
    "        )\n",
    "    ]\n",
    "    expected_steps = expected.get(\"steps\") or []\n",
```

```
    "    for index, (custom_step, reference_step, update) in
enumerate(\n",
    "        zip(custom[\"steps\"], reference[\"steps\"],
scenario[\"updates\"]), start=1\n",
    "    ):\n",
    "        step_info = expected_steps[index - 1] if index <=
len(expected_steps) else {}\n",
    "        ambiguous = global_ambiguous |
set(step_info.get(\"ambiguous_border_ids\", []))\n",
    "        row = _compare_snapshots(\n",
    "            f\"{kind}/{name}\",\n",
    "            index,\n",
    "            update[\"op\"],\n",
    "            custom_step,\n",
    "            reference_step,\n",
    "            ignore_ids=ambiguous,\n",
    "            eps=scenario[\"eps\"],\n",
    "            min_pts=scenario[\"min_pts\"],\n",
    "        )\n",
    "        row[\"point_id\"] = update[\"point_id\"]\n",
    "        row[\"verified_case\"] = step_info.get(\"case\", \"\")\n",
    "        rows.append(row)\n",
    "    return rows\n",
    "\n",
    "\n",
    "def _stream_rows(\n",
    "    stream: str,\n",
    "    raw_dir: Path = RAW,\n",
    "    max_steps: int | None = None,\n",
    ") -> list[dict[str, Any]]:\n",
    "    stream_dir = raw_dir / stream\n",
    "    d0, updates, params, _ = read_stream(stream_dir)\n",
    "    ambiguous = ambiguous_by_step(params, updates)\n",
    "    custom_frames = stream_snapshots(IncrementalDBSCAN, stream_dir,
max_steps=max_steps)\n",
    "    reference_frames = stream_snapshots(RefitDBSCAN, stream_dir,
max_steps=max_steps)\n",
    "    if len(custom_frames) != len(reference_frames):\n",
    "        raise ValueError(f\"{stream}: custom và sklearn có số step
khác nhau\")\n",
    "\n",
    "    eps = float(params.get(\"eps\", params.get(\"eps_km\")))\n",
    "    rows = []\n",
    "    for custom_frame, reference_frame in zip(custom_frames,
reference_frames):\n",
    "        step = int(custom_frame[\"step\"])\n",
    "        row = _compare_snapshots(\n",
    "            stream,\n",
    "            step,\n",
    "            custom_frame[\"op\"],\n",
    "            custom_frame[\"snapshot\"],\n",
    "            reference_frame[\"snapshot\"],\n",
    "            ignore_ids=set(ambiguous.get(step, [])),\n",
    "            eps=eps,\n",
```

```
    "            min_pts=int(params[\"minpts\"]),\n",
    "        )\n",
    "        row[\"point_id\"] = custom_frame[\"point_id\"]\n",
    "        row[\"verified_case\"] = custom_frame.get(\"case\") or
\"\"\n",
    "        rows.append(row)\n",
    "    return rows\n",
    "\n",
    "\n",
    "def run_equivalence_suite(\n",
    "    raw_dir: Path = RAW,\n",
    "    *,\n",
    "    include_scenarios: bool = True,\n",
    "    include_streams: bool = True,\n",
    "    max_stream_steps: int | None = None,\n",
    ") -> pd.DataFrame:\n",
    "    \"\"\"Đối chiếu kết quả phân cụm ban đầu và từng bước cập nhật
được chọn.\"\"\"\n",
    "    rows: list[dict[str, Any]] = []\n",
    "    if include_scenarios:\n",
    "        for kind in (\"incremental_examples\",):\n",
    "            for name in list_scenarios(kind, raw_dir):\n",
    "                rows.extend(_scenario_rows(kind, name, raw_dir))\n",
    "    if include_streams:\n",
    "        for stream in STREAMS:\n",
    "            rows.extend(_stream_rows(stream, raw_dir,
max_stream_steps))\n",
    "    return pd.DataFrame(rows)\n",
    "\n",
    "\n",
    "def write_equivalence_results(\n",
    "    output_path: Path = ROOT / \"results\" /
\"equivalence_results.csv\",\n",
    "    *,\n",
    "    max_stream_steps: int | None = None,\n",
    ") -> pd.DataFrame:\n",
    "    \"\"\"Thực hiện toàn bộ quá trình đối chiếu và lưu kết quả từng
lần phân cụm, cập nhật vào file CSV.\"\"\"\n",
    "    results =
run_equivalence_suite(max_stream_steps=max_stream_steps)\n",
    "    output_path.parent.mkdir(parents=True, exist_ok=True)\n",
    "    results.to_csv(output_path, index=False, encoding=\"utf-8\",
lineterminator=\"\\n\")\n",
    "    return results\n",
    "\n",
    "\n",
    "def main() -> int:\n",
    "    results = write_equivalence_results()\n",
    "    failed = results.loc[~results[\"equivalent\"]]\n",
    "    print(f\"Đã ghi {len(results)} kết quả so sánh vào
results/equivalence_results.csv\")\n",
    "    if failed.empty:\n",
```

```
    "        print(\"Tất cả trạng thái của Incremental DBSCAN đều khớp
với kết quả chạy lại DBSCAN bằng scikit-learn trên toàn bộ dữ
liệu.\")\n",
    "        return 0\n",
    "    print(f\"Phát hiện {len(failed)}  kết quả so sánh không trùng
khớp, kiểm tra file CSV để xem chi tiết từng trường hợp và bước cập
nhật.\")\n",
    "    return 1\n",
    "\n",
    "\n",
    "if __name__ == \"__main__\":\n",
    "    raise SystemExit(main())\n"
   ]
  },
  {
   "cell_type": "markdown",
   "id": "dc532c85",
   "metadata": {},
   "source": [
    "#### evaluation_library.py"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": null,
   "id": "a082639f",
   "metadata": {
    "vscode": {
     "languageId": "plaintext"
    }
   },
   "outputs": [],
   "source": [
    "from __future__ import annotations\n",
    "\n",
    "from typing import Any, Iterable\n",
    "\n",
    "from .metrics import _prepare_inputs\n",
    "\n",
    "\n",
    "def silhouette_samples(\n",
    "    data: Iterable[Iterable[float]],\n",
    "    labels: Iterable[Any],\n",
    "    *,\n",
    "    noise_label: Any = -1,\n",
    ") -> list[float | None]:\n",
    "    \"\"\"Tính hệ số Silhouette theo khoảng cách Euclid cho từng
điểm dữ liệu.\n",
    "\n",
    "    Các điểm nhiễu không tham gia tính toán và được biểu diễn bằng
'None'\n",
    "    trong danh sách kết quả, đồng thời giữ nguyên thứ tự đầu
vào.\n",
    "    \"\"\"\n",
```

```
    "    from sklearn.metrics import silhouette_samples as
sklearn_silhouette_samples\n",
    "\n",
    "    rows, label_list = _prepare_inputs(data, labels)\n",
    "    included = [index for index, label in enumerate(label_list) if
label != noise_label]\n",
    "    scores = sklearn_silhouette_samples(\n",
    "        [rows[index] for index in included],\n",
    "        [label_list[index] for index in included],\n",
    "        metric=\"euclidean\",\n",
    "    )\n",
    "    result: list[float | None] = [None] * len(rows)\n",
    "    for index, score in zip(included, scores):\n",
    "        result[index] = float(score)\n",
    "    return result\n",
    "\n",
    "\n",
    "def silhouette_score(\n",
    "    data: Iterable[Iterable[float]],\n",
    "    labels: Iterable[Any],\n",
    "    *,\n",
    "    noise_label: Any = -1,\n",
    ") -> float:\n",
    "    \"\"\"Tính hệ số Silhouette trung bình sau khi loại các điểm
nhiễu.\"\"\"\n",
    "    from sklearn.metrics import silhouette_score as
sklearn_silhouette_score\n",
    "\n",
    "    rows, label_list = _prepare_inputs(data, labels)\n",
    "    included = [index for index, label in enumerate(label_list) if
label != noise_label]\n",
    "    return float(\n",
    "        sklearn_silhouette_score(\n",
    "            [rows[index] for index in included],\n",
    "            [label_list[index] for index in included],\n",
    "            metric=\"euclidean\",\n",
    "        )\n",
    "    )\n",
    "\n",
    "\n",
    "def evaluate_library_metrics(\n",
    "    data: Iterable[Iterable[float]],\n",
    "    predicted_labels: Iterable[Any],\n",
    "    *,\n",
    "    true_labels: Iterable[Any] | None = None,\n",
    "    noise_label: Any = -1,\n",
    ") -> dict[str, float | int]:\n",
    "    \"\"\"Tính các chỉ số đánh giá chất lượng phân cụm bằng thư
viện.\n",
    "\n",
    "    Silhouette loại trừ điểm nhiễu. ARI sử dụng toàn bộ quan
sát,\n",
    "    trong đó 'noise_label' được xem như một nhãn dự đoán thông
thường.\n",
```

```
    "    Quy ước này giúp nhận diện sai lệch khi phát hiện nhiễu trong
quá trình\n",
    "    so sánh kết quả dự đoán với nhãn thực tế.\n",
    "    \"\"\"\n",
    "    from sklearn.metrics import adjusted_mutual_info_score,
adjusted_rand_score\n",
    "\n",
    "    rows, predicted = _prepare_inputs(data, predicted_labels)\n",
    "    included = [index for index, label in enumerate(predicted) if
label != noise_label]\n",
    "    result: dict[str, float | int] = {\n",
    "        \"silhouette_score\": silhouette_score(rows, predicted,
noise_label=noise_label),\n",
    "        \"n_clusters\": len({predicted[index] for index in
included}),\n",
    "        \"n_noise\": len(predicted) - len(included),\n",
    "    }\n",
    "\n",
    "    if true_labels is not None:\n",
    "        truth = list(true_labels)\n",
    "        if len(truth) != len(predicted):\n",
    "            raise ValueError(\"true_labels và predicted_labels phải
có cùng số phần tử\")\n",
    "        result[\"adjusted_rand_index\"] =
float(adjusted_rand_score(truth, predicted))\n",
    "        result[\"adjusted_mutual_information\"] = float(\n",
    "            adjusted_mutual_info_score(truth, predicted)\n",
    "        )\n",
    "    return result\n"
   ]
  },
  {
   "cell_type": "markdown",
   "id": "c67c2e80",
   "metadata": {},
   "source": [
    "#### metrics.py"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": null,
   "id": "81f7f4de",
   "metadata": {
    "vscode": {
     "languageId": "plaintext"
    }
   },
   "outputs": [],
   "source": [
    "from __future__ import annotations\n",
    "\n",
    "import math\n",
    "from collections import Counter, defaultdict\n",
```

```
    "from typing import Any, Iterable\n",
    "\n",
    "\n",
    "def _prepare_inputs(\n",
    "    data: Iterable[Iterable[float]], labels: Iterable[Any]\n",
    ") -> tuple[list[tuple[float, ...]], list[Any]]:\n",
    "    \"\"\"Kiểm tra tính hợp lệ và chuẩn hóa dữ liệu đặc trưng cùng
nhãn cụm.\"\"\"\n",
    "    try:\n",
    "        rows = [tuple(float(value) for value in row) for row in
data]\n",
    "    except (TypeError, ValueError) as exc:\n",
    "        raise ValueError(\"data phải là ma trận số hai chiều\") from
exc\n",
    "\n",
    "    try:\n",
    "        label_list = list(labels)\n",
    "    except TypeError as exc:\n",
    "        raise ValueError(\"labels phải là một dãy nhãn\") from
exc\n",
    "\n",
    "    if not rows:\n",
    "        raise ValueError(\"data không được rỗng\")\n",
    "    if len(rows) != len(label_list):\n",
    "        raise ValueError(\"data và labels phải có cùng số phần
tử\")\n",
    "    if not rows[0]:\n",
    "        raise ValueError(\"mỗi điểm phải có ít nhất một đặc
trưng\")\n",
    "    if any(len(row) != len(rows[0]) for row in rows):\n",
    "        raise ValueError(\"các điểm phải có cùng số chiều\")\n",
    "    if any(not math.isfinite(value) for row in rows for value in
row):\n",
    "        raise ValueError(\"data chỉ được chứa giá trị hữu hạn\")\n",
    "    try:\n",
    "        for label in label_list:\n",
    "            hash(label)\n",
    "    except TypeError as exc:\n",
    "        raise ValueError(\"mỗi nhãn phải là giá trị có thể băm\")
from exc\n",
    "    return rows, label_list\n",
    "\n",
    "\n",
    "def silhouette_samples(\n",
    "    data: Iterable[Iterable[float]],\n",
    "    labels: Iterable[Any],\n",
    "    *,\n",
    "    noise_label: Any = -1,\n",
    ") -> list[float | None]:\n",
    "    \"\"\"Tính hệ số Silhouette theo khoảng cách Euclid cho từng
điểm dữ liệu.\n",
    "\n",
    "    Các điểm nhiễu không tham gia tính toán và được biểu diễn bằng
'None'\n",
```

```
    "    trong danh sách kết quả, đồng thời giữ nguyên thứ tự đầu
vào.\n",
    "    \"\"\"\n",
    "    rows, label_list = _prepare_inputs(data, labels)\n",
    "    included = [index for index, label in enumerate(label_list) if
label != noise_label]\n",
    "    cluster_sizes = Counter(label_list[index] for index in
included)\n",
    "    cluster_labels = list(cluster_sizes)\n",
    "\n",
    "    if len(cluster_labels) < 2:\n",
    "        raise ValueError(\"Silhouette cần ít nhất hai cụm sau khi bỏ
noise\")\n",
    "    if len(included) <= len(cluster_labels):\n",
    "        raise ValueError(\"Silhouette cần nhiều điểm hơn số
cụm\")\n",
    "\n",
    "    result: list[float | None] = [None] * len(rows)\n",
    "    for index in included:\n",
    "        own_label = label_list[index]\n",
    "        own_size = cluster_sizes[own_label]\n",
    "        if own_size == 1:\n",
    "            result[index] = 0.0\n",
    "            continue\n",
    "\n",
    "        distance_sums: dict[Any, float] = defaultdict(float)\n",
    "        for other_index in included:\n",
    "            if other_index == index:\n",
    "                continue\n",
    "            other_label = label_list[other_index]\n",
    "            distance_sums[other_label] += math.dist(rows[index],
rows[other_index])\n",
    "\n",
    "        within_cluster_distance = distance_sums[own_label] /
(own_size - 1)\n",
    "        nearest_other_cluster_distance = min(\n",
    "            distance_sums[other_label] / size\n",
    "            for other_label, size in cluster_sizes.items()\n",
    "            if other_label != own_label\n",
    "        )\n",
    "        denominator = max(within_cluster_distance,
nearest_other_cluster_distance)\n",
    "        result[index] = (\n",
    "            0.0\n",
    "            if denominator == 0.0\n",
    "            else (nearest_other_cluster_distance -
within_cluster_distance) / denominator\n",
    "        )\n",
    "    return result\n",
    "\n",
    "\n",
    "def silhouette_score(\n",
    "    data: Iterable[Iterable[float]],\n",
    "    labels: Iterable[Any],\n",
```

```
    "    *,\n",
    "    noise_label: Any = -1,\n",
    ") -> float:\n",
    "    \"\"\"Return the mean silhouette after excluding noise
points.\"\"\"\n",
    "    samples = silhouette_samples(data, labels,
noise_label=noise_label)\n",
    "    included_samples = [value for value in samples if value is not
None]\n",
    "    return sum(included_samples) / len(included_samples)\n"
   ]
  },
  {
   "cell_type": "markdown",
   "id": "e49db6e5",
   "metadata": {},
   "source": [
    "#### engine.py"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": null,
   "id": "00f6aa1f",
   "metadata": {
    "vscode": {
     "languageId": "plaintext"
    }
   },
   "outputs": [],
   "source": [
    "\"\"\"IncrementalEngine: IncrementalDBSCAN của nhóm cộng thêm
snapshot() và vài thuộc tính đếm.\n",
    "\n",
    "Lớp không đổi thuật toán, chỉ đọc self.points. Nhờ có snapshot() mà
replay_stream, run_stream (baseline) và bộ vẽ (src/visualization)\n",
    "chạy với thuật toán thật giống hệt như đã chạy với RefitDBSCAN.
Không dùng sklearn.\n",
    "\"\"\"\n",
    "import numpy as np\n",
    "\n",
    "from src.clustering.incremental_dbscan import IncrementalDBSCAN\n",
    "from src.core.data_point import PointType\n",
    "\n",
    "# mã loại điểm giống baseline.sklearn_dbscan.PointType và
src.visualization.plotter\n",
    "TYPE_CODE = {PointType.UNCLASSIFIED: 0, PointType.NOISE: 1,
PointType.BORDER: 2, PointType.CORE: 3}\n",
    "\n",
    "\n",
    "class IncrementalEngine(IncrementalDBSCAN):\n",
    "    \"\"\"IncrementalDBSCAN với snapshot() theo thứ tự id tăng dần;
cluster_id None hoặc -1 đều ghi là nhiễu (-1).\"\"\"\n",
    "\n",
```

```
    "    def snapshot(self):\n",
    "        ids = sorted(self.points)\n",
    "        pts = [self.points[i] for i in ids]\n",
    "        dim = pts[0].dimension if pts else 0\n",
    "        coords = np.vstack([p.coords for p in pts]) if pts else
np.empty((0, dim))\n",
    "        labels = np.array([-1 if p.cluster_id is None else
int(p.cluster_id) for p in pts], dtype=int)\n",
    "        types = np.array([TYPE_CODE[p.point_type] for p in pts],
dtype=int)\n",
    "        return {\"ids\": np.array(ids, dtype=int), \"coords\":
coords, \"labels\": labels, \"point_types\": types}\n",
    "\n",
    "    @property\n",
    "    def n_points(self):\n",
    "        return len(self.points)\n",
    "\n",
    "    @property\n",
    "    def n_clusters(self):\n",
    "        return len({p.cluster_id for p in self.points.values() if
p.cluster_id is not None and p.cluster_id >= 0})\n",
    "\n",
    "    @property\n",
    "    def n_noise(self):\n",
    "        return sum(1 for p in self.points.values() if p.cluster_id
is None or p.cluster_id < 0)\n"
   ]
  },
  {
   "cell_type": "markdown",
   "id": "8ac83f69",
   "metadata": {},
   "source": [
    "#### update_cases.py"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": null,
   "id": "b443dc12",
   "metadata": {
    "vscode": {
     "languageId": "plaintext"
    }
   },
   "outputs": [],
   "source": [
    "\"\"\"Đo một lần cập nhật từ hai snapshot (trước, sau): UpdSeed, số
điểm đổi trạng thái, phạm vi xử lý lại của phép xóa. numpy thuần, không
sklearn.\n",
    "\n",
    "Snapshot là dict ids, coords, labels, point_types (mã 3 lõi, 2 biên,
1 nhiễu) như IncrementalEngine.snapshot() hay RefitDBSCAN.snapshot().\n",
```

```
    "Định nghĩa UpdSeed theo Ester et al. (1998), như mô tả trong
docs/oop_design.md:\n",
    "  insert: các điểm lõi SAU update nằm trong Eps của một điểm vừa
thành lõi (kể cả điểm chèn nếu nó là lõi);\n",
    "  delete: các điểm lõi SAU update nằm trong Eps của một điểm vừa mất
tính lõi (kể cả điểm xóa nếu nó từng là lõi).\n",
    "\"\"\"\n",
    "import numpy as np\n",
    "\n",
    "from src.visualization.plotter import align_labels\n",
    "\n",
    "CORE = 3\n",
    "\n",
    "\n",
    "def _ids(snap):\n",
    "    return np.asarray(snap[\"ids\"]).tolist()\n",
    "\n",
    "\n",
    "def core_ids(snap):\n",
    "    return {i for i, t in zip(_ids(snap),
np.asarray(snap[\"point_types\"]).tolist()) if t == CORE}\n",
    "\n",
    "\n",
    "def _coords_of(snap, wanted):\n",
    "    pos = {i: k for k, i in enumerate(_ids(snap))}\n",
    "    return np.asarray(snap[\"coords\"], dtype=float)[[pos[i] for i
in wanted]]\n",
    "\n",
    "\n",
    "def within_eps(snap, centers, eps, candidates=None):\n",
    "    \"\"\"Tập id của `snap` (hoặc của candidates) cách ít nhất một
điểm trong `centers` (mảng tọa độ) không quá eps, biên bao hàm.\"\"\"\n",
    "    ids = _ids(snap)\n",
    "    coords = np.asarray(snap[\"coords\"], dtype=float)\n",
    "    centers = np.asarray(centers, dtype=float).reshape(-1,
coords.shape[1] if coords.size else 1)\n",
    "    if not len(ids) or not len(centers):\n",
    "        return set()\n",
    "    keep = np.ones(len(ids), dtype=bool) if candidates is None else
np.array([i in candidates for i in ids])\n",
    "    near = np.zeros(len(ids), dtype=bool)\n",
    "    for c in centers:\n",
    "        near |= np.sqrt(((coords - c) ** 2).sum(axis=1)) <= eps\n",
    "    return {i for i, k in zip(ids, near & keep) if k}\n",
    "\n",
    "\n",
    "def update_seeds(before, after, op, eps):\n",
    "    \"\"\"UpdSeed của một update (tập id), cùng tập điểm đổi tính
lõi (vừa thành lõi khi insert, vừa mất tính lõi khi delete).\"\"\"\n",
    "    cb, ca = core_ids(before), core_ids(after)\n",
    "    if op == \"insert\":\n",
    "        changed = sorted(ca - cb)\n",
    "        centers = _coords_of(after, changed)\n",
    "    else:\n",
```

```
    "        changed = sorted(cb - ca)\n",
    "        centers = _coords_of(before, changed)\n",
    "    return within_eps(after, centers, eps, candidates=ca),
set(changed)\n",
    "\n",
    "\n",
    "def changed_points(before, after):\n",
    "    \"\"\"Id có mặt ở cả hai snapshot mà loại điểm hoặc cụm thay
đổi. Cụm của `after` được khớp về `before` (align_labels), nên đánh số
lại không tính,\n",
    "    còn hai cụm nhập làm một thì các điểm của cụm bị nhập tính là
đổi.\"\"\"\n",
    "    lb = dict(zip(_ids(before),
np.asarray(before[\"labels\"]).tolist()))\n",
    "    tb = dict(zip(_ids(before),
np.asarray(before[\"point_types\"]).tolist()))\n",
    "    la = align_labels(before, after).tolist()\n",
    "    return {i for i, l, t in zip(_ids(after), la,
np.asarray(after[\"point_types\"]).tolist())\n",
    "            if i in lb and (lb[i] != l or tb[i] != t)}\n",
    "\n",
    "\n",
    "def delete_scope(before, point_id, eps):\n",
    "    \"\"\"Ước lượng số điểm thuộc cụm cũ giao với vùng ảnh hưởng của
điểm xóa.\n",
    "\n",
    "    Đây là quy mô cụm có thể bị tác động theo định nghĩa Affected
Objects của\n",
    "    paper, không phải số điểm thực tế được duyệt trong bước BFS
UpdSeedDel.\n",
    "    \"\"\"\n",
    "    labels = dict(zip(_ids(before),
np.asarray(before[\"labels\"]).tolist()))\n",
    "    near = within_eps(before, _coords_of(before, [point_id]),
eps)\n",
    "    touched = {labels[i] for i in near if labels[i] >= 0}\n",
    "    return sum(1 for l in labels.values() if l in touched)\n",
    "\n",
    "\n",
    "def describe_update(before, after, op, point_id, eps):\n",
    "    \"\"\"Tóm tắt kích thước dữ liệu, UpdSeed, điểm đổi và vùng có
thể bị ảnh hưởng.\"\"\"\n",
    "    seeds, changed_core = update_seeds(before, after, op, eps)\n",
    "    row = {\"n_before\": len(_ids(before)), \"n_after\":
len(_ids(after)), \"n_updseed\": len(seeds), \"n_core_changed\":
len(changed_core),\n",
    "           \"n_changed\": len(changed_points(before, after) -
{point_id})}\n",
    "    row[\"n_potentially_affected\"] = delete_scope(before, point_id,
eps) if op == \"delete\" else np.nan\n",
    "    return row\n"
   ]
  },
  {
```

```
   "cell_type": "markdown",
   "id": "a97d5fd5",
   "metadata": {},
   "source": [
    "\"\"\"Chuẩn hóa thủ công và chuyển tọa độ cho luồng Incremental
DBSCAN.\"\"\"\n",
    "from .scaler import ManualScaler\n",
    "from .stream_scaling import apply_frozen_scaling, feature_columns,
stream_coordinates\n",
    "\n",
    "__all__ = [\"ManualScaler\", \"apply_frozen_scaling\",
\"feature_columns\", \"stream_coordinates\"]\n"
   ]
  },
  {
   "cell_type": "markdown",
   "id": "0d96f25a",
   "metadata": {},
   "source": [
    "#### scaler.py"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": null,
   "id": "ebcdd6a2",
   "metadata": {
    "vscode": {
     "languageId": "plaintext"
    }
   },
   "outputs": [],
   "source": [
    "\"\"\"ManualScaler: chuẩn hóa đặc trưng bằng numpy thuần (không
sklearn, không scipy).\n",
    "\n",
    "Giao diện theo docs/oop_design.md: fit, transform, fit_transform,
inverse_transform.\n",
    "\n",
    "Hai phương pháp, cùng một công thức  z = (x - center) / scale:\n",
    "    method='standard': center = mean, scale = std tổng thể (ddof=0,
giống StandardScaler)\n",
    "    method='minmax'  : center = min,  scale = max - min     (giống
MinMaxScaler)\n",
    "Cột có độ lệch bằng 0 (hằng số) nhận scale = 1 để tránh chia cho
0.\n",
    "\n",
    "Quy tắc cho Incremental DBSCAN: tham số phải đứng yên trong suốt
luồng. Điểm mới đi qua transform với đúng tham số đó; không fit lại,\n",
    "nếu không tọa độ của các điểm cũ sẽ đổi và nhãn cụm đã tính không
còn nhất quán. Luồng benchmark dùng hằng số chuẩn hóa cố định trong\n",
    "`params.json`, tính trên toàn bộ luồng mô phỏng để mọi thuật toán
nhận cùng tọa độ. Vì hằng số này có dùng dữ liệu ở các bước sau D0,\n",
```

```
    "nó phục vụ so sánh có kiểm soát, không mô phỏng triển khai online
không biết trước dữ liệu tương lai. Xem stream_scaling.py.\n",
    "\"\"\"\n",
    "import numpy as np\n",
    "\n",
    "_METHODS = (\"standard\", \"minmax\")\n",
    "\n",
    "\n",
    "class ManualScaler:\n",
    "    def __init__(self, method=\"standard\"):\n",
    "        if method not in _METHODS:\n",
    "            raise ValueError(f\"method phải là một trong {_METHODS},
nhận được {method!r}\")\n",
    "        self.method = method\n",
    "        self.center_ = None       # mean (standard) hoặc min
(minmax), shape (d,)\n",
    "        self.scale_ = None        # std (standard) hoặc range
(minmax), đã thay 0 bằng 1\n",
    "        self.feature_names_ = None  # tên cột nếu fit từ
DataFrame\n",
    "        self.n_samples_ = 0\n",
    "\n",
    "    # ---------- kiểm tra đầu vào ----------\n",
    "    def _as_2d(self, data, fitting):\n",
    "        \"\"\"Trả về (mảng float 2-D, có phải vector 1-D không). Báo
lỗi rõ nếu có NaN/inf.\"\"\"\n",
    "        names = list(data.columns) if hasattr(data, \"columns\")
else None\n",
    "        arr = np.asarray(data, dtype=float)\n",
    "        was_1d = arr.ndim == 1\n",
    "        if was_1d:\n",
    "            if fitting:\n",
    "                raise ValueError(\"fit cần ma trận 2-D (n_mẫu,
n_cột); nhận được vector 1-D\")\n",
    "            arr = arr[None, :]\n",
    "        if arr.ndim != 2:\n",
    "            raise ValueError(f\"dữ liệu phải 2-D, nhận được
{arr.ndim}-D\")\n",
    "        if fitting and arr.shape[0] == 0:\n",
    "            raise ValueError(\"không thể fit trên 0 dòng\")\n",
    "        bad = ~np.isfinite(arr)\n",
    "        if bad.any():\n",
    "            cols = np.flatnonzero(bad.any(axis=0))\n",
    "            label = [names[c] if names else int(c) for c in
cols]\n",
    "            raise ValueError(\n",
    "                f\"dữ liệu có {int(bad.sum())} ô NaN/inf ở cột
{label}; \"\n",
    "                \"hãy xử lý ở bước làm sạch (điền khuyết) trước khi
chuẩn hóa\")\n",
    "        if fitting and names is not None:\n",
    "            self.feature_names_ = names\n",
    "        return arr, was_1d\n",
    "\n",
```

```
    "    def _check_fitted(self):\n",
    "        if self.center_ is None:\n",
    "            raise RuntimeError(\"ManualScaler chưa fit; gọi
fit(data) trước\")\n",
    "\n",
    "    # ---------- giao diện chính ----------\n",
    "    def fit(self, data):\n",
    "        x, _ = self._as_2d(data, fitting=True)\n",
    "        if self.method == \"standard\":\n",
    "            center = x.mean(axis=0)\n",
    "            scale = x.std(axis=0)                    # ddof=0\n",
    "        else:\n",
    "            center = x.min(axis=0)\n",
    "            scale = x.max(axis=0) - center\n",
    "        self.center_ = center\n",
    "        self.scale_ = np.where(scale == 0.0, 1.0, scale)\n",
    "        self.n_samples_ = x.shape[0]\n",
    "        return self\n",
    "\n",
    "    def transform(self, data):\n",
    "        \"\"\"Dùng tham số đã fit. Vector 1-D (một điểm mới) trả về
vector 1-D.\"\"\"\n",
    "        self._check_fitted()\n",
    "        x, was_1d = self._as_2d(data, fitting=False)\n",
    "        if x.shape[1] != self.center_.shape[0]:\n",
    "            raise ValueError(f\"cần {self.center_.shape[0]} cột,
nhận được {x.shape[1]}\")\n",
    "        z = (x - self.center_) / self.scale_\n",
    "        return z[0] if was_1d else z\n",
    "\n",
    "    def fit_transform(self, data):\n",
    "        return self.fit(data).transform(data)\n",
    "\n",
    "    def inverse_transform(self, data):\n",
    "        self._check_fitted()\n",
    "        z, was_1d = self._as_2d(data, fitting=False)\n",
    "        if z.shape[1] != self.center_.shape[0]:\n",
    "            raise ValueError(f\"cần {self.center_.shape[0]} cột,
nhận được {z.shape[1]}\")\n",
    "        x = z * self.scale_ + self.center_\n",
    "        return x[0] if was_1d else x\n",
    "\n",
    "    # ---------- lưu và nạp tham số (JSON) ----------\n",
    "    def to_dict(self):\n",
    "        self._check_fitted()\n",
    "        return {\"method\": self.method,\n",
    "                \"center\": self.center_.tolist(),\n",
    "                \"scale\": self.scale_.tolist(),\n",
    "                \"feature_names\": self.feature_names_,\n",
    "                \"n_samples\": int(self.n_samples_)}\n",
    "\n",
    "    @classmethod\n",
    "    def from_dict(cls, params):\n",
```

```
    "        \"\"\"Tạo scaler từ dict (ví dụ đọc từ JSON). Với
'standard': center = mean, scale = std.\"\"\"\n",
    "        obj = cls(params.get(\"method\", \"standard\"))\n",
    "        obj.center_ = np.asarray(params[\"center\"],
dtype=float)\n",
    "        scale = np.asarray(params[\"scale\"], dtype=float)\n",
    "        if obj.center_.shape != scale.shape or obj.center_.ndim !=
1:\n",
    "            raise ValueError(\"center và scale phải là hai vector 1-
D cùng độ dài\")\n",
    "        obj.scale_ = np.where(scale == 0.0, 1.0, scale)\n",
    "        obj.feature_names_ = params.get(\"feature_names\")\n",
    "        obj.n_samples_ = int(params.get(\"n_samples\", 0))\n",
    "        return obj\n",
    "\n",
    "    def __repr__(self):\n",
    "        state = \"đã fit\" if self.center_ is not None else \"chưa
fit\"\n",
    "        return f\"ManualScaler(method={self.method!r}, {state})\"\n"
   ]
  },
  {
   "cell_type": "markdown",
   "id": "451987cb",
   "metadata": {},
   "source": [
    "#### stream_scaling.py"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": null,
   "id": "220966ed",
   "metadata": {
    "vscode": {
     "languageId": "plaintext"
    }
   },
   "outputs": [],
   "source": [
    "\"\"\"Dùng chung phép chuẩn hóa cố định cho notebook, benchmark và
kiểm tra luồng.\n",
    "\n",
    "`apply_frozen_scaling` log10-transform amount_vnd rồi áp mean/std
trong params.json. Các hằng số được tính từ toàn bộ dữ liệu\n",
    "mô phỏng, gồm cả các điểm xuất hiện sau D0. Vì vậy benchmark bảo đảm
mọi thuật toán nhận cùng tọa độ nhưng không mô phỏng\n",
    "một hệ thống online không biết trước dữ liệu tương lai.\n",
    "\"\"\"\n",
    "import numpy as np\n",
    "\n",
    "from .scaler import ManualScaler\n",
    "\n",
    "LOG_PREFIX = \"log10_\"\n",
```

```
    "\n",
    "\n",
    "def feature_columns(d0, params=None):\n",
    "    \"\"\"Tên các đặc trưng theo thứ tự chuẩn hóa trong params; hỗ
trợ cả D0 chỉ có cột số.\"\"\"\n",
    "    if params and params.get(\"frozen_scaling\"):\n",
    "        return frozen_columns(params)\n",
    "    excluded = {\"point_id\", \"ref_label\", \"segment_id\"}\n",
    "    return [column for column in d0.columns if column not in
excluded]\n",
    "\n",
    "\n",
    "def frozen_columns(params):\n",
    "    \"\"\"Tên cột THÔ theo thứ tự của frozen_scaling (bỏ tiền tố
log10_), hoặc None nếu stream không có frozen_scaling.\"\"\"\n",
    "    fs = params.get(\"frozen_scaling\")\n",
    "    return None if fs is None else [c[len(LOG_PREFIX):] if
c.startswith(LOG_PREFIX) else c for c in fs[\"columns\"]]\n",
    "\n",
    "\n",
    "def apply_frozen_scaling(params, frame):\n",
    "    \"\"\"Tọa độ chuẩn hóa theo hằng số cố định của stream, không
fit lại.\n",
    "\n",
    "    params: dict đọc từ params.json (cần khóa frozen_scaling gồm
columns, mean, std).\n",
    "    frame: DataFrame có các cột thô (d0.csv, hoặc các dòng
updates.csv). Cột có tiền tố log10_ trong frozen_scaling\n",
    "    được lấy log10 từ cột thô cùng tên (amount_vnd); thứ tự cột kết
quả theo params, không theo frame.\n",
    "    Trả mảng float (len(frame), d). Báo lỗi rõ nếu thiếu
frozen_scaling, thiếu cột, hoặc số tiền không dương.\n",
    "    \"\"\"\n",
    "    fs = params.get(\"frozen_scaling\")\n",
    "    if fs is None:\n",
    "        raise ValueError(\"params không có frozen_scaling (stream
ATM dùng stream_coordinates)\")\n",
    "    raw_cols = frozen_columns(params)\n",
    "    missing = [c for c in raw_cols if c not in frame.columns]\n",
    "    if missing:\n",
    "        raise ValueError(f\"cột frozen_scaling {fs['columns']} không
khớp cột stream: thiếu {missing}\")\n",
    "    x = frame[raw_cols].to_numpy(dtype=float).copy()\n",
    "    for j, c in enumerate(fs[\"columns\"]):\n",
    "        if c.startswith(LOG_PREFIX):\n",
    "            if (x[:, j] <= 0).any():\n",
    "                raise ValueError(f\"cột {raw_cols[j]} có giá trị
không dương, không lấy log10 được\")\n",
    "            x[:, j] = np.log10(x[:, j])\n",
    "    scaler = ManualScaler.from_dict({\"method\": \"standard\",
\"center\": fs[\"mean\"], \"scale\": fs[\"std\"],\n",
    "                                     \"feature_names\":
fs[\"columns\"]})\n",
    "    return scaler.transform(x)\n",
```

```
    "\n",
    "\n",
    "def stream_coordinates(params, frame, feature_cols):\n",
    "    \"\"\"Tọa độ đi vào engine cho BẤT KỲ stream nào: frozen_scaling
nếu có, nếu không thì cột thô (ATM, theo km).\n",
    "\n",
    "    feature_cols: kết quả của feature_columns(d0); với stream thẻ
chúng phải khớp frozen_scaling (cùng tập, cùng thứ tự)\n",
    "    để d0 và updates dùng đúng cùng một ma trận cột.\n",
    "    \"\"\"\n",
    "    if params.get(\"frozen_scaling\") is None:\n",
    "        return frame[feature_cols].to_numpy(dtype=float).copy()\n",
    "    if frozen_columns(params) != list(feature_cols):\n",
    "        raise ValueError(f\"cột frozen_scaling
{params['frozen_scaling']['columns']} không khớp cột stream
{list(feature_cols)}\")\n",
    "    return apply_frozen_scaling(params, frame)\n"
   ]
  },
  {
   "cell_type": "markdown",
   "id": "fd189ba4",
   "metadata": {},
   "source": [
    "#### gallery.py"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": null,
   "id": "0b46f71d",
   "metadata": {
    "vscode": {
     "languageId": "plaintext"
    }
   },
   "outputs": [],
   "source": [
    "\"\"\"Tạo dữ liệu cho gallery các ví dụ insert và delete.\n",
    "\n",
    "    cases = build_gallery_cases(\"incremental_examples\",
RefitDBSCAN)          # engine_factory(eps, min_pts)\n",
    "    fig = ClusterVisualizer().plot_case_gallery(cases,
n_pair_cols=2)\n",
    "\"\"\"\n",
    "from .scenarios import list_scenarios, load_scenario,
run_scenario\n",
    "\n",
    "# (tiêu đề đậm, chú thích một dòng) cho 8 ví dụ, theo 7 trường hợp
của báo cáo (Potential Split có hai ví dụ A và B)\n",
    "EXAMPLE_CAPTIONS = {\n",
    "    \"r02_1_noise\": (\"1. Noise\", \"chèn p, p là nhiễu\"),\n",
    "    \"r02_2_creation\": (\"2. Creation\", \"chèn p, tạo cụm
mới\"),\n",
```

```
    "    \"r02_3_absorption\": (\"3. Absorption\", \"chèn p, p vào cụm
cũ\"),\n",
    "    \"r02_4_merge\": (\"4. Merge\", \"chèn p, hai cụm hợp
nhất\"),\n",
    "    \"r03_1_removal\": (\"5. Removal\", \"xóa p, cụm biến
mất\"),\n",
    "    \"r03_2_reduction\": (\"6. Reduction\", \"xóa p, cụm thu
nhỏ\"),\n",
    "    \"r03_3a_split\": (\"7a. Split\", \"xóa p, cụm tách đôi\"),\n",
    "    \"r03_3b_nonsplit\": (\"7b. Non-split\", \"xóa p, cụm vẫn
liền\"),\n",
    "}\n",
    "\n",
    "def build_gallery_cases(kind, engine_factory, names=None):\n",
    "    \"\"\"Chạy các ví dụ cập nhật trên engine và trả danh sách case
cho ClusterVisualizer.plot_case_gallery.\n",
    "\n",
    "    Mỗi case: before, after, update, seed_ids, eps, header, note,
name. names chọn bớt kịch bản (mặc định tất cả).\"\"\"\n",
    "    if kind != \"incremental_examples\":\n",
    "        raise ValueError(\"Gallery chỉ hỗ trợ các ví dụ trong
data/raw/incremental_examples\")\n",
    "    out = []\n",
    "    for name in names or list_scenarios(kind):\n",
    "        sc = load_scenario(kind, name)\n",
    "        run = run_scenario(engine_factory, sc)\n",
    "        header, note = EXAMPLE_CAPTIONS[name]\n",
    "        out.append({\"name\": name, \"before\": run[\"before\"],
\"after\": run[\"after\"], \"update\": run[\"update\"], \"seed_ids\":
run[\"seed_ids\"],\n",
    "                    \"eps\": sc[\"eps\"] if
run[\"before\"][\"coords\"].shape[1] == 2 else None, \"header\": header,
\"note\": note,\n",
    "                    \"expected\": sc[\"expected\"]})\n",
    "    return out\n"
   ]
  },
  {
   "cell_type": "markdown",
   "id": "c7f747b9",
   "metadata": {},
   "source": [
    "#### plotter.py"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": null,
   "id": "7621bea0",
   "metadata": {
    "vscode": {
     "languageId": "plaintext"
    }
   },
```

```
   "outputs": [],
   "source": [
    "\"\"\"Trực quan hóa cụm (D07): ClusterVisualizer vẽ MỘT ảnh chụp
(snapshot) của mô hình gom cụm.\n",
    "\n",
    "Snapshot là dict gồm bốn khóa, cùng độ dài n:\n",
    "    ids          id của điểm (số nguyên hoặc chuỗi; chỉ cần phân
biệt được các điểm)\n",
    "    coords       ma trận (n, 2) tọa độ để vẽ trực tiếp\n",
    "    labels       nhãn cụm, -1 là nhiễu\n",
    "    point_types  loại điểm: 3 = lõi (core), 2 = biên (border), 1 =
nhiễu (noise), 0 = chưa phân loại (có thể bỏ, để None)\n",
    "Đây đúng là dạng `RefitDBSCAN.snapshot()` của baseline/, nên lớp này
chạy được ngay hôm nay và chạy được với IncrementalDBSCAN\n",
    "thật của nhóm mà không phải sửa gì, miễn là đổi MỘT chỗ: hàm
`snapshot_from_model` ở dưới (xem docstring của nó).\n",
    "\n",
    "Quy ước vẽ: lõi = hình tròn đặc, biên = tam giác, nhiễu = dấu x xám;
mỗi cụm một màu (bảng Okabe-Ito, an toàn cho người mù màu),\n",
    "vùng tô nhạt của cụm = hợp các hình tròn bán kính Eps quanh các điểm
lõi (khi biết Eps) hoặc bao lồi (khi không biết).\n",
    "Điểm được chèn = ngôi sao, điểm bị xóa = dấu X (vẽ tại vị trí cũ),
UpdSeed = vòng tròn viền đen, Eps = đường tròn nét đứt.\n",
    "\n",
    "Chỉ dùng numpy và matplotlib; không dùng sklearn hay scipy (quy ước
của src/). Không đặt backend: tự chọn Agg khi chạy không màn hình.\n",
    "\"\"\"\n",
    "from contextlib import contextmanager\n",
    "\n",
    "import numpy as np\n",
    "import matplotlib\n",
    "import matplotlib.pyplot as plt\n",
    "from matplotlib import patheffects\n",
    "from matplotlib.collections import PatchCollection\n",
    "from matplotlib.lines import Line2D\n",
    "from matplotlib.patches import Circle, Polygon\n",
    "\n",
    "# Mã loại điểm, trùng với baseline.sklearn_dbscan.PointType (và thứ
tự trong docs/oop_design.md).\n",
    "UNCLASSIFIED, NOISE, BORDER, CORE = 0, 1, 2, 3\n",
    "_TYPE_BY_NAME = {\"UNCLASSIFIED\": UNCLASSIFIED, \"NOISE\": NOISE,
\"BORDER\": BORDER, \"CORE\": CORE}\n",
    "\n",
    "# Okabe-Ito (an toàn cho người mù màu) thêm hai màu của Paul Tol;
quá 9 cụm thì màu lặp lại (chấp nhận được khi các cụm cách xa nhau).\n",
    "PALETTE = [\"#0072B2\", \"#E69F00\", \"#009E73\", \"#D55E00\",
\"#56B4E9\", \"#CC79A7\", \"#882255\", \"#332288\", \"#999933\"]\n",
    "NOISE_COLOR = \"#8C8C8C\"\n",
    "_HALO = [patheffects.withStroke(linewidth=2.2,
foreground=\"white\")]        # viền trắng quanh chữ để đọc được khi nằm
trên đường kẻ\n",
    "INK = \"#1A1A1A\"\n",
    "\n",
```

```
    "# Giao diện cho báo cáo A4 (Word hoặc PDF): phông DejaVu Sans có sẵn
trong matplotlib và đủ dấu tiếng Việt.\n",
    "REPORT_RC = {\n",
    "    \"font.family\": \"DejaVu Sans\", \"font.size\": 8,
\"axes.titlesize\": 9, \"axes.labelsize\": 8,\n",
    "    \"xtick.labelsize\": 7, \"ytick.labelsize\": 7,
\"legend.fontsize\": 7, \"figure.dpi\": 100, \"savefig.dpi\": 200,\n",
    "    \"axes.spines.top\": False, \"axes.spines.right\": False,
\"axes.grid\": False, \"pdf.fonttype\": 42, \"svg.fonttype\":
\"none\",\n",
    "}\n",
    "\n",
    "\n",
    "@contextmanager\n",
    "def report_style():\n",
    "    \"\"\"Áp REPORT_RC trong một khối with rồi trả lại đúng các khóa
đó. KHÔNG dùng plt.rc_context: nó khôi phục cả khóa `interactive`, và
trong\n",
    "    Jupyter (backend inline) việc tạo hình đầu tiên bên trong
rc_context làm mọi hình sau đó không còn được hiển thị.\"\"\"\n",
    "    old = {k: matplotlib.rcParams[k] for k in REPORT_RC}\n",
    "    matplotlib.rcParams.update(REPORT_RC)\n",
    "    try:\n",
    "        yield\n",
    "    finally:\n",
    "        matplotlib.rcParams.update(old)\n",
    "\n",
    "\n",
    "# ---------------------------------------------------------------------
snapshot\n",
    "def _type_code(value):\n",
    "    \"\"\"Mã loại điểm (0 đến 3) từ số nguyên, IntEnum hoặc Enum
(đọc theo TÊN CORE, BORDER, NOISE, UNCLASSIFIED nếu có).\"\"\"\n",
    "    name = getattr(value, \"name\", None)\n",
    "    if isinstance(name, str) and name.upper() in _TYPE_BY_NAME:\n",
    "        return _TYPE_BY_NAME[name.upper()]\n",
    "    if isinstance(value, str):\n",
    "        return _TYPE_BY_NAME.get(value.upper(), UNCLASSIFIED)\n",
    "    return int(value)\n",
    "\n",
    "\n",
    "def make_snapshot(ids, coords, labels, point_types=None):\n",
    "    \"\"\"Chuẩn hóa bốn dãy thành snapshot (mảng numpy, bản sao).
point_types None được điền 0 (chưa phân loại).\"\"\"\n",
    "    coords = np.asarray(coords, dtype=float)\n",
    "    if coords.ndim == 1:\n",
    "        coords = coords.reshape(-1, 1) if coords.size else
coords.reshape(0, 2)\n",
    "    if coords.ndim != 2:\n",
    "        raise ValueError(\"coords cần ma trận 2-D (n_điểm,
n_chiều)\")\n",
    "    n = len(coords)\n",
    "    ids = np.array(list(ids))\n",
    "    labels = np.array(labels, dtype=int).ravel()\n",
```

```
    "    types = np.zeros(n, dtype=int) if point_types is None else
np.array([_type_code(t) for t in point_types], dtype=int)\n",
    "    if not (len(ids) == len(labels) == len(types) == n):\n",
    "        raise ValueError(f\"độ dài không khớp: ids {len(ids)},
coords {n}, labels {len(labels)}, point_types {len(types)}\")\n",
    "    return {\"ids\": ids, \"coords\": coords.copy(), \"labels\":
labels, \"point_types\": types}\n",
    "\n",
    "\n",
    "def snapshot_from_model(model):\n",
    "    \"\"\"ADAPTER DUY NHẤT từ một mô hình gom cụm sang snapshot.
Nhóm chưa chốt tên phương thức, nên mọi thay đổi chỉ sửa ở đây.\n",
    "\n",
    "    Thử lần lượt:\n",
    "      1. model.snapshot() trả dict có ids, coords, labels,
point_types (RefitDBSCAN hiện nay; nên cài cùng giao diện này cho lớp
thật);\n",
    "      2. model.points hoặc model._points: dãy (hoặc dict id -> điểm)
các đối tượng giống DataPoint trong docs/oop_design.md, mỗi\n",
    "         điểm có .id, .coords, .cluster_id (None nghĩa là nhiễu) và
.point_type (số, IntEnum hoặc Enum có tên CORE, BORDER, NOISE).\n",
    "    Nếu cả hai không có, báo lỗi nêu rõ cần cài gì. Một dict đã đúng
dạng snapshot cũng được chấp nhận.\n",
    "    \"\"\"\n",
    "    if isinstance(model, dict):\n",
    "        return make_snapshot(model[\"ids\"], model[\"coords\"],
model[\"labels\"], model.get(\"point_types\"))\n",
    "    if callable(getattr(model, \"snapshot\", None)):\n",
    "        snap = model.snapshot()\n",
    "        return make_snapshot(snap[\"ids\"], snap[\"coords\"],
snap[\"labels\"], snap.get(\"point_types\"))\n",
    "    pts = getattr(model, \"points\", None)\n",
    "    pts = getattr(model, \"_points\", None) if pts is None else
pts\n",
    "    if pts is not None:\n",
    "        pts = list(pts.values()) if isinstance(pts, dict) else
list(pts)\n",
    "        if not pts:\n",
    "            return make_snapshot([], np.empty((0, 2)), [], [])\n",
    "        labels = [-1 if p.cluster_id is None else int(p.cluster_id)
for p in pts]\n",
    "        return make_snapshot([p.id for p in pts],
np.vstack([np.asarray(p.coords, float).ravel() for p in pts]),
labels,\n",
    "                             [_type_code(p.point_type) for p in
pts])\n",
    "    raise TypeError(\"không lấy được snapshot: mô hình cần phương
thức snapshot() trả dict (ids, coords, labels, point_types) \"\n",
    "                    \"hoặc thuộc tính points gồm các đối tượng có
id, coords, cluster_id, point_type\")\n",
    "\n",
    "\n",
    "def type_counts(snap):\n",
```

```
    "    \"\"\"Đếm core, border, noise, other (điểm thuộc cụm nhưng chưa
phân loại) và số cụm của một snapshot.\"\"\"\n",
    "    lab, typ = snap[\"labels\"], snap[\"point_types\"]\n",
    "    noise = lab == -1\n",
    "    core, border = (typ == CORE) & ~noise, (typ == BORDER) &
~noise\n",
    "    return {\"core\": int(core.sum()), \"border\":
int(border.sum()), \"noise\": int(noise.sum()),\n",
    "            \"other\": int((~noise & ~core & ~border).sum()),
\"clusters\": int(len(set(lab.tolist()) - {-1})), \"n\":
int(len(lab))}\n",
    "\n",
    "\n",
    "def align_labels(reference, snapshot, min_new_label=0):\n",
    "    \"\"\"Đổi nhãn của `snapshot` để cụm giữ MÀU của cụm trùng nhiều
điểm nhất trong `reference` (khớp một-một, tham lam theo số điểm
chung).\n",
    "\n",
    "    reference, snapshot: snapshot hoặc dict có ids và labels. Cụm
không khớp được nhận nhãn mới bắt đầu từ\n",
    "    max(nhãn của reference, min_new_label - 1) + 1. Nhiễu (-1) giữ
nguyên. Trả mảng nhãn mới (bản sao).\n",
    "    \"\"\"\n",
    "    ref = dict(zip(np.asarray(reference[\"ids\"]).tolist(),
np.asarray(reference[\"labels\"]).tolist()))\n",
    "    ids, labels = np.asarray(snapshot[\"ids\"]).tolist(),
np.asarray(snapshot[\"labels\"], dtype=int)\n",
    "    counts = {}\n",
    "    for i, l in zip(ids, labels.tolist()):\n",
    "        r = ref.get(i)\n",
    "        if l != -1 and r is not None and r != -1:\n",
    "            counts[(l, r)] = counts.get((l, r), 0) + 1\n",
    "    mapping, taken = {}, set()\n",
    "    for (l, r), _ in sorted(counts.items(), key=lambda kv: (-kv[1],
kv[0][1], kv[0][0])):\n",
    "        if l not in mapping and r not in taken:\n",
    "            mapping[l] = r\n",
    "            taken.add(r)\n",
    "    nxt = max([max(ref.values(), default=-1), min_new_label - 1]) +
1\n",
    "    out = labels.copy()\n",
    "    for l in sorted(set(labels.tolist()) - {-1}):\n",
    "        if l not in mapping:\n",
    "            mapping[l] = nxt\n",
    "            nxt += 1\n",
    "    for l, m in mapping.items():\n",
    "        out[labels == l] = m\n",
    "    return out\n",
    "\n",
    "\n",
    "def _convex_hull(pts):\n",
    "    \"\"\"Bao lồi (monotone chain, numpy thuần) của các điểm 2-D;
trả mảng các đỉnh ngược chiều kim đồng hồ (có thể < 3 đỉnh).\"\"\"\n",
    "    p = np.unique(np.asarray(pts, dtype=float), axis=0)\n",
```

```
    "    if len(p) <= 2:\n",
    "        return p\n",
    "\n",
    "    def cross(o, a, b):\n",
    "        return (a[0] - o[0]) * (b[1] - o[1]) - (a[1] - o[1]) * (b[0]
- o[0])\n",
    "    lower, upper = [], []\n",
    "    for q in p.tolist():\n",
    "        while len(lower) >= 2 and cross(lower[-2], lower[-1], q) <=
0:\n",
    "            lower.pop()\n",
    "        lower.append(q)\n",
    "    for q in p.tolist()[::-1]:\n",
    "        while len(upper) >= 2 and cross(upper[-2], upper[-1], q) <=
0:\n",
    "            upper.pop()\n",
    "        upper.append(q)\n",
    "    return np.array(lower[:-1] + upper[:-1])\n",
    "\n",
    "\n",
    "def label_colors(labels):\n",
    "    \"\"\"Màng màu (n, 3) theo nhãn: nhiễu xám, cụm lấy PALETTE[nhãn
% độ dài].\"\"\"\n",
    "    from matplotlib.colors import to_rgb\n",
    "    return np.array([to_rgb(NOISE_COLOR if l < 0 else PALETTE[int(l)
% len(PALETTE)]) for l in labels]).reshape(-1, 3)\n",
    "\n",
    "\n",
    "def _normalize_updates(update):\n",
    "    if update is None:\n",
    "        return []\n",
    "    items = [update] if isinstance(update, dict) else
list(update)\n",
    "    for u in items:\n",
    "        if u.get(\"op\") not in (\"insert\", \"delete\"):\n",
    "            raise ValueError(f\"update cần khóa op là 'insert' hoặc
'delete', nhận được {u.get('op')!r}\")\n",
    "    return items\n",
    "\n",
    "\n",
    "# ---------------------------------------------------------------------
lớp vẽ\n",
    "class ClusterVisualizer:\n",
    "    \"\"\"Vẽ snapshot gom cụm. Mọi phương thức nhận snapshot (dict)
hoặc một mô hình có snapshot_from_model chấp nhận.\"\"\"\n",
    "\n",
    "    def __init__(self, point_size=38):\n",
    "        self.point_size = float(point_size)\n",
    "\n",
    "    # -- chuẩn bị\n",
    "    @staticmethod\n",
    "    def as_snapshot(obj):\n",
    "        return snapshot_from_model(obj)\n",
    "\n",
```

```
    "    @staticmethod\n",
    "    def _limits(arrays, extra=None, pad=0.12, min_aspect=0.6):\n",
    "        pts = np.vstack([a for a in arrays if len(a)] or
[np.zeros((1, 2))])\n",
    "        lo, hi = pts.min(axis=0), pts.max(axis=0)\n",
    "        if extra is not None and len(extra):\n",
    "            lo, hi = np.minimum(lo, extra[:, :2].min(axis=0)),
np.maximum(hi, extra[:, :2].max(axis=0))\n",
    "        span = np.maximum(hi - lo, 1e-9)\n",
    "        span = np.where(span < 1e-6, 1.0, span)\n",
    "        m = span * pad\n",
    "        xl, yl = [lo[0] - m[0], hi[0] + m[0]], [lo[1] - m[1], hi[1]
+ m[1]]\n",
    "        w, h = xl[1] - xl[0], yl[1] - yl[0]\n",
    "        if h < min_aspect * w:                      # hình quá dẹt:
nới trục y đối xứng để panel không còn một sợi\n",
    "            c = (yl[0] + yl[1]) / 2\n",
    "            yl = [c - min_aspect * w / 2, c + min_aspect * w /
2]\n",
    "        elif w < min_aspect * h:\n",
    "            c = (xl[0] + xl[1]) / 2\n",
    "            xl = [c - min_aspect * h / 2, c + min_aspect * h /
2]\n",
    "        return tuple(xl), tuple(yl)\n",
    "\n",
    "    def _to_2d(self, snap, basis=None):\n",
    "        \"\"\"Báo lỗi nếu dữ liệu nhiều chiều.\"\"\"\n",
    "        if snap[\"coords\"].shape[1] != 2:\n",
    "            raise ValueError(\"Trực quan hóa chỉ hỗ trợ dữ liệu
2D.\")\n",
    "        return snap, None\n",
    "\n",
    "    # -- vẽ lõi\n",
    "    def _draw(self, ax, snap, eps=None, highlight_ids=None,
seed_ids=None, marks=None, fill=\"auto\", annotate_ids=None,\n",
    "              legend=True, axis_labels=None, compact=False,
limits=None, title=None, proj=None, colors=None):\n",
    "        s = self.point_size * (0.42 if compact else 1.0)\n",
    "        xy, lab, typ, ids = snap[\"coords\"], snap[\"labels\"],
snap[\"point_types\"], snap[\"ids\"]\n",
    "        idx = {i: k for k, i in enumerate(ids.tolist())}\n",
    "        eps_ok = eps is not None\n",
    "        mode = (\"eps\" if eps_ok else \"hull\") if fill == \"auto\"
else fill\n",
    "        if mode == \"eps\" and not eps_ok:\n",
    "            mode = \"hull\"\n",
    "        noise = lab == -1\n",
    "        core, border = (typ == CORE) & ~noise, (typ == BORDER) &
~noise\n",
    "        other = ~noise & ~core & ~border\n",
    "        col = label_colors(lab) if colors is None else colors\n",
    "\n",
    "        if mode == \"eps\":\n",
    "            patches, fcs = [], []\n",
```

```
    "            for k in np.flatnonzero(core | (other & ~noise)):\n",
    "                patches.append(Circle(xy[k], eps))\n",
    "                fcs.append(col[k])\n",
    "            if patches:\n",
    "                ax.add_collection(PatchCollection(patches,
facecolors=fcs, alpha=0.16, linewidths=0, zorder=1))\n",
    "        elif mode == \"hull\":\n",
    "            for l in sorted(set(lab.tolist()) - {-1}):\n",
    "                h = _convex_hull(xy[lab == l])\n",
    "                if len(h) >= 3:\n",
    "                    c = PALETTE[l % len(PALETTE)]\n",
    "                    ax.add_patch(Polygon(h, closed=True,
facecolor=c, alpha=0.14, edgecolor=c, linewidth=0.8, zorder=1))\n",
    "\n",
    "        def scat(mask, marker, gid, **kw):\n",
    "            if mask.any():\n",
    "                sc = ax.scatter(xy[mask, 0], xy[mask, 1],
marker=marker, s=s, zorder=3, **kw)\n",
    "                sc.set_gid(gid)\n",
    "        scat(core, \"o\", \"core\", c=col[core], edgecolors=INK,
linewidths=0.45)\n",
    "        scat(border, \"^\", \"border\", c=col[border],
edgecolors=INK, linewidths=0.45)\n",
    "        scat(other, \"o\", \"point\", c=col[other], edgecolors=INK,
linewidths=0.45)\n",
    "        scat(noise, \"x\", \"noise\", c=NOISE_COLOR, linewidths=0.9
if not compact else 0.7)\n",
    "\n",
    "        shown = {\"seed\": False, \"insert\": False, \"delete\":
False, \"highlight\": False}\n",
    "        for name, group in ((\"seed_ids\", seed_ids),
(\"highlight_ids\", highlight_ids), (\"annotate_ids\",
annotate_ids)):\n",
    "            missing = [i for i in (group or []) if i not in idx]\n",
    "            if missing:\n",
    "                raise KeyError(f\"{name} có id không nằm trong
snapshot: {missing} (id của snapshot: {list(idx)[:6]}...)\")\n",
    "        if seed_ids is not None:\n",
    "            seeds = [idx[i] for i in seed_ids]\n",
    "            if seeds:\n",
    "                ax.scatter(xy[seeds, 0], xy[seeds, 1], s=s * 3.2,
facecolors=\"none\", edgecolors=INK, linewidths=1.0 if compact else
1.3,\n",
    "                           zorder=4).set_gid(\"seed\")\n",
    "                shown[\"seed\"] = True\n",
    "        circle_centres = []\n",
    "        if highlight_ids is not None:\n",
    "            hl = [idx[i] for i in highlight_ids]\n",
    "            if hl:\n",
    "                ax.scatter(xy[hl, 0], xy[hl, 1], s=s * 3.8,
facecolors=\"none\", edgecolors=INK, linewidths=1.4,
zorder=5).set_gid(\"highlight\")\n",
    "                circle_centres += [xy[k] for k in hl]\n",
    "                shown[\"highlight\"] = True\n",
```

```
    "        pending = []                                          # nhãn điểm
cập nhật, ghi sau khi biết khung trục\n",
    "        for m in marks or []:\n",
    "            ghost = m.get(\"ghost\", False)\n",
    "            star = m[\"kind\"] == \"insert\"\n",
    "            ax.scatter([m[\"xy\"][0]], [m[\"xy\"][1]], marker=\"*\"
if star else \"X\", s=s * (6.5 if star else 4.0),\n",
    "                       facecolors=\"none\" if ghost else INK,
edgecolors=INK if ghost else \"white\",\n",
    "                       linewidths=1.0 if ghost else 0.7,
zorder=6).set_gid(f\"{m['kind']}{'_ghost' if ghost else ''}\")\n",
    "            if not ghost:\n",
    "                circle_centres.append(np.asarray(m[\"xy\"],
float))\n",
    "                shown[m[\"kind\"]] = True\n",
    "            pending.append((None if m.get(\"label\") is None else
str(m[\"label\"]), np.asarray(m[\"xy\"], float), not ghost))\n",
    "        rings = [c for c in circle_centres if not
any(np.array_equal(c, q[1]) for q in pending)]      # vòng của điểm nhấn
mạnh\n",
    "        for i in annotate_ids or []:\n",
    "            ax.annotate(str(i), xy[idx[i]], xytext=(3, 3),
textcoords=\"offset points\", fontsize=6.5, color=INK, zorder=7,
path_effects=_HALO)\n",
    "\n",
    "        extra = None\n",
    "        if eps_ok and circle_centres:\n",
    "            c = np.vstack(circle_centres)\n",
    "            extra = np.vstack([c - eps, c + eps])\n",
    "        xl, yl = limits if limits is not None else
self._limits([xy], extra)\n",
    "        ax.set_xlim(*xl)\n",
    "        ax.set_ylim(*yl)\n",
    "        # các điểm cập nhật nằm sát nhau (nối chuỗi, mỗi bước dưới
9% khung trục) thành một nhóm: một nhãn chung đặt ở điểm trên cùng\n",
    "        # bên phải, và chỉ điểm cập nhật sau cùng của nhóm có vòng
Eps, để nhiều bước liền nhau không chồng chữ và chồng vòng\n",
    "        span = max(xl[1] - xl[0], yl[1] - yl[0])\n",
    "        groups = []\n",
    "        for item in pending:\n",
    "            near = [g for g in groups if any(np.linalg.norm(o[1] -
item[1]) < 0.09 * span for o in g)]\n",
    "            merged = [item] + [o for g in near for o in g]\n",
    "            groups = [g for g in groups if all(g is not h for h in
near)] + [merged]\n",
    "        order = {id(it): k for k, it in enumerate(pending)}\n",
    "        offsets = [(3.5, 3.5), (3.5, -9.0), (-11.0, 3.5), (-11.0, -
9.0)]\n",
    "        for k, g in enumerate(groups):\n",
    "            g = sorted(g, key=lambda it: order[id(it)])\n",
    "            texts = [it[0] for it in g if it[0] is not None]\n",
    "            if texts:\n",
    "                top = max(g, key=lambda it: (it[1][1],
it[1][0]))[1]\n",
```

```
    "                ax.annotate(\", \".join(dict.fromkeys(texts)), top,
xytext=offsets[k % len(offsets)], textcoords=\"offset points\",\n",
    "                            fontsize=6.5 if compact else 8,
fontweight=\"bold\", color=INK, zorder=7, path_effects=_HALO)\n",
    "            solid = [it for it in g if it[2]]\n",
    "            if solid:\n",
    "                rings.append(solid[-1][1])\n",
    "        if eps_ok:\n",
    "            for c in rings:\n",
    "                ax.add_patch(Circle(c, eps, fill=False,
linestyle=(0, (3, 2)), edgecolor=INK, linewidth=0.8, zorder=5))\n",
    "        ax.set_aspect(\"equal\", adjustable=\"box\")\n",
    "        if compact:\n",
    "            ax.set_anchor(\"N\")\n",
    "            ax.set_xticks([])\n",
    "            ax.set_yticks([])\n",
    "            for sp in ax.spines.values():\n",
    "                sp.set_visible(True)\n",
    "                sp.set_linewidth(0.6)\n",
    "                sp.set_color(\"#999999\")\n",
    "        else:\n",
    "            labs = axis_labels or (\"x\", \"y\")\n",
    "            ax.set_xlabel(labs[0])\n",
    "            ax.set_ylabel(labs[1])\n",
    "        if title:\n",
    "            ax.set_title(title, pad=3 if compact else 6,
fontsize=6.8 if compact else None)\n",
    "        if legend:\n",
    "            self._legend(ax, snap, shown, eps_ok and
bool(circle_centres))\n",
    "        return ax\n",
    "\n",
    "    def _legend(self, ax, snap, shown, with_eps, fig=None,
counts=True, loc=\"upper center\", anchor=(0.5, -0.14), ncol=3,
present=None):\n",
    "        \"\"\"present (khi counts=False): dict core, border, noise
cho biết loại điểm nào có mặt trong hình; thiếu thì liệt kê đủ ba
loại.\"\"\"\n",
    "        c = type_counts(snap) if counts else None\n",
    "        has = present or {\"core\": True, \"border\": True,
\"noise\": True}\n",
    "        ink = {\"markerfacecolor\": \"#BBBBBB\",
\"markeredgecolor\": INK, \"markeredgewidth\": 0.5, \"linestyle\":
\"\"}\n",
    "\n",
    "        def lab(text, key):\n",
    "            return f\"{text}: {c[key]}\" if c is not None else
text\n",
    "        handles = []\n",
    "        if (c is None and has[\"core\"]) or (c is not None and
c[\"core\"]):\n",
    "            handles.append(Line2D([], [], marker=\"o\",
markersize=6, label=lab(\"Điểm lõi (core)\", \"core\"), **ink))\n",
```

```
    "        if (c is None and has[\"border\"]) or (c is not None and
c[\"border\"]):\n",
    "            handles.append(Line2D([], [], marker=\"^\",
markersize=6, label=lab(\"Điểm biên (border)\", \"border\"), **ink))\n",
    "        if c is not None and c[\"other\"]:\n",
    "            handles.append(Line2D([], [], marker=\"o\",
markersize=6, label=lab(\"Điểm thuộc cụm\", \"other\"), **ink))\n",
    "        if (c is None and has[\"noise\"]) or (c is not None and
c[\"noise\"]):\n",
    "            handles.append(Line2D([], [], marker=\"x\",
color=NOISE_COLOR, markersize=6, linestyle=\"\", markeredgewidth=1.0,\n",
    "                                  label=lab(\"Nhiễu (noise)\",
\"noise\")))\n",
    "        if shown.get(\"seed\"):\n",
    "            handles.append(Line2D([], [], marker=\"o\",
markersize=8, markerfacecolor=\"none\", markeredgecolor=INK,
linestyle=\"\", label=\"UpdSeed\"))\n",
    "        if shown.get(\"insert\"):\n",
    "            handles.append(Line2D([], [], marker=\"*\",
markersize=9, markerfacecolor=INK, markeredgecolor=\"white\",
linestyle=\"\", label=\"Điểm chèn (insert)\"))\n",
    "        if shown.get(\"delete\"):\n",
    "            handles.append(Line2D([], [], marker=\"X\",
markersize=7, markerfacecolor=INK, markeredgecolor=\"white\",
linestyle=\"\", label=\"Điểm xóa (delete)\"))\n",
    "        if shown.get(\"highlight\"):\n",
    "            handles.append(Line2D([], [], marker=\"o\",
markersize=8, markerfacecolor=\"none\", markeredgecolor=INK,
linestyle=\"\", label=\"Điểm được nhấn mạnh\"))\n",
    "        if with_eps:\n",
    "            handles.append(Line2D([], [], color=INK, linestyle=(0,
(3, 2)), linewidth=0.9, label=\"Vòng Eps\"))\n",
    "        if c is not None:\n",
    "            handles.append(Line2D([], [], linestyle=\"\",
label=f\"Số cụm: {c['clusters']}\"))\n",
    "        target = fig if fig is not None else ax\n",
    "        target.legend(handles=handles, loc=loc,
bbox_to_anchor=anchor, ncol=ncol, frameon=False, handletextpad=0.4,
columnspacing=1.2)\n",
    "\n",
    "    # -- API công khai\n",
    "    def plot_2d(self, snapshot, ax=None, eps=None,
highlight_ids=None, title=None, *, fill=\"auto\", seed_ids=None,\n",
    "                annotate_ids=None, legend=True, axis_labels=None,
compact=False, limits=None):\n",
    "        \"\"\"Vẽ một snapshot lên `ax` (tạo mới nếu None) và trả về
Axes.\n",
    "\n",
    "        snapshot: dict (ids, coords, labels, point_types) hoặc mô
hình (xem snapshot_from_model); chỉ vẽ dữ liệu 2D.\n",
    "        eps: bán kính Eps (đơn vị tọa độ); có Eps thì vùng tô cụm là
hợp các hình tròn Eps quanh điểm lõi và điểm nhấn mạnh có vòng Eps\n",
    "        highlight_ids: id cần nhấn mạnh (vòng tròn đậm). seed_ids:
id vẽ vòng UpdSeed.\n",
```

```
    "        fill: 'auto' | 'eps' | 'hull' | None. legend=True hiện chú
thích có số lượng core, border, noise và số cụm. Mỗi loại điểm là một\n",
    "        PathCollection có gid 'core', 'border', 'noise', 'point' (số
điểm = len(get_offsets())).\"\"\"\n",
    "        snap = snapshot_from_model(snapshot)\n",
    "        with report_style():\n",
    "            if ax is None:\n",
    "                _, ax = plt.subplots(figsize=(4.6, 4.6) if legend
else (4.0, 3.6))\n",
    "            s2, proj = self._to_2d(snap)\n",
    "            return self._draw(ax, s2, eps, highlight_ids, seed_ids,
None, fill, annotate_ids, legend, axis_labels, compact, limits, title,
proj)\n",
    "\n",
    "    def _marks(self, update, before, after):\n",
    "        \"\"\"Danh sách dấu cập nhật cho (before, after): mỗi dấu
{kind, xy, label, ghost} theo panel, xy theo tọa độ gốc d
chiều.\"\"\"\n",
    "        mb, ma = [], []\n",
    "        for u in _normalize_updates(update):\n",
    "            pid = u.get(\"point_id\")\n",
    "            xy = u.get(\"coords\")\n",
    "            if u[\"op\"] == \"insert\":\n",
    "                if xy is None:\n",
    "                    k = [j for j, i in
enumerate(after[\"ids\"].tolist()) if i == pid]\n",
    "                    if not k:\n",
    "                        raise ValueError(f\"không thấy điểm chèn
{pid!r} trong snapshot sau\")\n",
    "                    xy = after[\"coords\"][k[0]]\n",
    "                mb.append({\"kind\": \"insert\", \"xy\":
np.asarray(xy, float), \"label\": pid, \"ghost\": True})\n",
    "                ma.append({\"kind\": \"insert\", \"xy\":
np.asarray(xy, float), \"label\": pid, \"ghost\": False})\n",
    "            else:\n",
    "                if xy is None:\n",
    "                    k = [j for j, i in
enumerate(before[\"ids\"].tolist()) if i == pid]\n",
    "                    if not k:\n",
    "                        raise ValueError(f\"không thấy điểm xóa
{pid!r} trong snapshot trước\")\n",
    "                    xy = before[\"coords\"][k[0]]\n",
    "                mb.append({\"kind\": \"delete\", \"xy\":
np.asarray(xy, float), \"label\": pid, \"ghost\": False})\n",
    "                ma.append({\"kind\": \"delete\", \"xy\":
np.asarray(xy, float), \"label\": None, \"ghost\": True})\n",
    "        return mb, ma\n",
    "\n",
    "    def plot_before_after(self, before, after, update=None,
eps=None, seed_ids=None, *, axes=None, titles=(\"Trước\", \"Sau\"),\n",
    "                          suptitle=None, legend=True, compact=False,
axis_labels=None, fill=\"auto\", match_colors=True):\n",
```

```
    "        \"\"\"Hai panel (trước, sau) trên cùng tọa độ và cùng màu
cụm (nhãn của `after` được khớp về `before`). Trả Figure (hoặc Figure của
axes).\n",
    "\n",
    "        update: dict {op, point_id, coords (tùy chọn)} hoặc danh
sách dict theo thứ tự (kịch bản nhiều dòng cho thấy trạng thái cuối).\n",
    "        op='insert': ngôi sao tại vị trí điểm, đặc ở panel sau, rỗng
ở panel trước. op='delete': dấu X đặc đè lên điểm ở panel trước,\n",
    "        rỗng ở panel sau (vị trí cũ). Vòng Eps nét đứt quanh dấu
đặc. seed_ids: id UpdSeed, vẽ vòng đen ở panel sau.\n",
    "        Cả hai trạng thái phải có tọa độ 2D. axes: cặp Axes có sẵn
(dùng cho bộ ảnh).\"\"\"\n",
    "        b, a = snapshot_from_model(before),
snapshot_from_model(after)\n",
    "        mb, ma = self._marks(update, b, a)\n",
    "        with report_style():\n",
    "            if axes is None:\n",
    "                fig, axes = plt.subplots(1, 2, figsize=(7.0, 3.9 if
legend else 3.4))\n",
    "            else:\n",
    "                fig = axes[0].figure\n",
    "            if b[\"coords\"].shape[1] != 2 or a[\"coords\"].shape[1]
!= 2:\n",
    "                raise ValueError(\"Hai trạng thái phải có tọa độ
2D.\")\n",
    "\n",
    "            basis = None\n",
    "            b2, pb = self._to_2d(b)\n",
    "            a2, pa = self._to_2d(a)\n",
    "            if match_colors:\n",
    "                a2 = dict(a2)\n",
    "                a2[\"labels\"] = align_labels(b, a)\n",
    "            extra = np.array([mk[\"xy\"] for mk in mb + ma]) if (mb
or ma) else None\n",
    "            lim_extra = None\n",
    "            if extra is not None and eps is not None and basis is
None:\n",
    "                lim_extra = np.vstack([extra - eps, extra +
eps])\n",
    "            elif extra is not None:\n",
    "                lim_extra = extra\n",
    "            limits = self._limits([b2[\"coords\"], a2[\"coords\"]],
lim_extra)\n",
    "            for ax, snap, marks, prx, seeds, ttl in ((axes[0], b2,
mb, pb, None, titles[0]), (axes[1], a2, ma, pa, seed_ids,
titles[1])):\n",
    "                c = type_counts(snap)\n",
    "                sub = f\"{c['clusters']} cụm, {c['noise']} nhiễu\"
if compact else f\"{c['n']} điểm, {c['clusters']} cụm, {c['noise']}
nhiễu\"\n",
    "                self._draw(ax, snap, eps, None, seeds, marks, fill,
None, False, axis_labels, compact, limits, f\"{ttl} · {sub}\", prx,\n",
    "
colors=label_colors(snap[\"labels\"]))\n",
```

```
    "            if legend:\n",
    "                shown = {\"seed\": bool(seed_ids), \"insert\":
any(m[\"kind\"] == \"insert\" for m in ma), \"delete\": any(m[\"kind\"]
== \"delete\" for m in mb)}\n",
    "                both = [type_counts(b), type_counts(a)]\n",
    "                present = {k: any(t[k] for t in both) for k in
(\"core\", \"border\", \"noise\")}\n",
    "                self._legend(None, a2, shown, eps is not None and
basis is None and bool(mb or ma), fig=fig, counts=False,\n",
    "                             anchor=(0.5, 0.0), ncol=4,
present=present)\n",
    "            if suptitle:\n",
    "                fig.suptitle(suptitle, fontsize=10,
fontweight=\"bold\")\n",
    "            if legend and axes is not None:\n",
    "                fig.subplots_adjust(bottom=0.2)\n",
    "            return fig\n",
    "\n",
    "    def plot_case_gallery(self, cases, n_pair_cols=2, width=7.5,
panel_gap=0.04, pair_gap=0.18, margin=0.18, header_h=0.46,\n",
    "                          legend_h=0.55, fill=\"hull\"):\n",
    "        \"\"\"Bộ ảnh nhiều kịch bản trước và sau (khổ rộng `width`
inch, vừa chiều ngang A4). Trả Figure.\n",
    "\n",
    "        cases: danh sách dict {before, after, update, seed_ids, eps,
header, note}; header in đậm trên mỗi cặp panel, note là dòng chú thích
nhỏ.\n",
    "        Mỗi hàng chứa n_pair_cols cặp (trước, sau); chú thích chung
ở cuối. fill mặc định 'hull' (bao lồi) vì vùng Eps của toy lớn hơn khoảng
cách\n",
    "        giữa các điểm và che hết hình khi panel nhỏ; truyền
fill='eps' để tô hợp hình tròn Eps.\"\"\"\n",
    "        n = len(cases)\n",
    "        rows = -(-n // n_pair_cols)\n",
    "        pw = (width - 2 * margin - (n_pair_cols - 1) * pair_gap -
n_pair_cols * panel_gap) / (2 * n_pair_cols)\n",
    "\n",
    "        def build(panel_hs):\n",
    "            \"\"\"Dựng Figure với chiều cao panel riêng từng hàng;
trả (fig, chiều cao thật của các panel theo hàng).\"\"\"\n",
    "            tops = [margin + sum(h + header_h + 0.12 for h in
panel_hs[:r]) for r in range(rows)]\n",
    "            height = margin + sum(h + header_h + 0.12 for h in
panel_hs) + legend_h\n",
    "            fig = plt.figure(figsize=(width, height))\n",
    "            used = [0.0] * rows\n",
    "            for k, case in enumerate(cases):\n",
    "                r, c = divmod(k, n_pair_cols)\n",
    "                x0 = margin + c * (2 * pw + panel_gap +
pair_gap)\n",
    "                ytop = height - tops[r]\n",
    "                axs = [fig.add_axes([(x0 + j * (pw + panel_gap)) /
width, (ytop - header_h - panel_hs[r]) / height, pw / width,\n",
```

```
    "                                        panel_hs[r] / height]) for j in
range(2)]\n",
    "                self.plot_before_after(case[\"before\"],
case[\"after\"], case.get(\"update\"), case.get(\"eps\"),
case.get(\"seed_ids\"),\n",
    "                                       axes=axs, legend=False,
compact=True, fill=fill)\n",
    "                for ax in axs:\n",
    "                    ax.apply_aspect()\n",
    "                    used[r] = max(used[r], ax.get_position().height
* height)\n",
    "                fig.text((x0 + pw + panel_gap / 2) / width, (ytop -
0.02) / height, case.get(\"header\", \"\"), ha=\"center\",
va=\"top\",\n",
    "                         fontsize=8, fontweight=\"bold\")\n",
    "                if case.get(\"note\"):\n",
    "                    fig.text((x0 + pw + panel_gap / 2) / width,
(ytop - 0.2) / height, case[\"note\"], ha=\"center\", va=\"top\",\n",
    "                             fontsize=6.4, color=\"#444444\")\n",
    "            ink = {\"markerfacecolor\": \"#BBBBBB\",
\"markeredgecolor\": INK, \"markeredgewidth\": 0.5, \"linestyle\":
\"\"}\n",
    "            entries = [(has[\"core\"], Line2D([], [], marker=\"o\",
markersize=5, label=\"Điểm lõi (core)\", **ink)),\n",
    "                       (has[\"border\"], Line2D([], [],
marker=\"^\", markersize=5, label=\"Điểm biên (border)\", **ink)),\n",
    "                       (has[\"noise\"], Line2D([], [], marker=\"x\",
color=NOISE_COLOR, markersize=5, linestyle=\"\", label=\"Nhiễu
(noise)\")),\n",
    "                       (has_ins, Line2D([], [], marker=\"*\",
markersize=10, markerfacecolor=INK, markeredgecolor=\"white\",
linestyle=\"\",\n",
    "                                        label=\"Điểm vừa chèn (sao
đặc)\")),\n",
    "                       (has_ins, Line2D([], [], marker=\"*\",
markersize=8, markerfacecolor=\"none\", markeredgecolor=INK,
linestyle=\"\",\n",
    "                                        label=\"Vị trí sắp chèn (sao
rỗng)\")),\n",
    "                       (has_del, Line2D([], [], marker=\"X\",
markersize=6, markerfacecolor=INK, markeredgecolor=\"white\",
linestyle=\"\",\n",
    "                                        label=\"Điểm bị xóa (X
đặc)\")),\n",
    "                       (has_del, Line2D([], [], marker=\"X\",
markersize=6, markerfacecolor=\"none\", markeredgecolor=INK,
linestyle=\"\",\n",
    "                                        label=\"Vị trí đã xóa (X
rỗng)\")),\n",
    "                       (has_seed, Line2D([], [], marker=\"o\",
markersize=7, markerfacecolor=\"none\", markeredgecolor=INK,
linestyle=\"\", label=\"UpdSeed\")),\n",
```

```
    "                        (has_eps, Line2D([], [], color=INK,
linestyle=(0, (3, 2)), linewidth=0.9, label=\"Vòng Eps quanh điểm cập
nhật\"))]\n",
    "            handles = [h for ok, h in entries if ok]\n",
    "            ncol = 4 if len(handles) <= 8 else 5\n",
    "            fig.legend(handles=handles, loc=\"lower center\",
bbox_to_anchor=(0.5, 0.0), ncol=ncol, frameon=False, fontsize=6.4,\n",
    "                       handletextpad=0.3, columnspacing=1.0)\n",
    "            fig.text(0.5, (legend_h - 0.1) / height, \"Màu = cụm
(giữ nguyên giữa trước và sau)\", ha=\"center\", va=\"bottom\",\n",
    "                     fontsize=6.4, color=\"#444444\")\n",
    "            return fig, used\n",
    "\n",
    "        counts = [type_counts(snapshot_from_model(c[k])) for c in
cases for k in (\"before\", \"after\")]\n",
    "        has = {k: any(t[k] for t in counts) for k in (\"core\",
\"border\", \"noise\")}\n",
    "        ups = [u for c in cases for u in
_normalize_updates(c.get(\"update\"))]\n",
    "        has_ins, has_del = any(u[\"op\"] == \"insert\" for u in
ups), any(u[\"op\"] == \"delete\" for u in ups)\n",
    "        has_seed = any(c.get(\"seed_ids\") for c in cases)\n",
    "        has_eps = bool(ups) and any(c.get(\"eps\") is not None for c
in cases)\n",
    "        with report_style():\n",
    "            fig, used = build([pw] * rows)          # lượt 1: đo
chiều cao thật của panel (tỉ lệ khung hình đều nhau), lượt 2: co khoảng
trống\n",
    "            plt.close(fig)\n",
    "            fig, _ = build([max(u, 0.3) + 0.02 for u in used])\n",
    "        return fig\n",
    "\n",
    "    @staticmethod\n",
    "    def save(fig, path, dpi=200):\n",
    "        \"\"\"Lưu Figure ra file (tạo thư mục nếu thiếu), nền trắng,
cắt lề thừa. Trả đường dẫn.\"\"\"\n",
    "        from pathlib import Path\n",
    "        path = Path(path)\n",
    "        path.parent.mkdir(parents=True, exist_ok=True)\n",
    "        with report_style():\n",
    "            fig.savefig(path, dpi=dpi, bbox_inches=\"tight\",
facecolor=\"white\")\n",
    "        return path\n"
   ]
  },
  {
   "cell_type": "markdown",
   "id": "27f6e529",
   "metadata": {},
   "source": [
    "#### reports.py"
   ]
  },
  {
```

```
   "cell_type": "code",
   "execution_count": null,
   "id": "6a2184f9",
   "metadata": {
    "vscode": {
     "languageId": "plaintext"
    }
   },
   "outputs": [],
   "source": [
    "\"\"\"Hình cho báo cáo nhận DataFrame hoặc mảng (D07, dùng lại cho
D09 và D10). Chỉ numpy, pandas, matplotlib. Mọi hàm trả Figure.\n",
    "\n",
    "plot_equivalence_timeline   ARI từng step so với re-fit toàn bộ (kỳ
vọng phẳng ở 1,0), đánh dấu step bị lệch\n",
    "plot_runtime_comparison     thời gian mỗi update: incremental so với
re-fit\n",
    "plot_affected_points        số điểm bị ảnh hưởng mỗi update\n",
    "plot_algorithm_contrast     KMeans, DBSCAN, HAC (hoặc thuật toán
khác) cạnh nhãn thật\n",
    "plot_k_distance             đồ thị k-distance chọn Eps, khoảng cách
tính bằng numpy\n",
    "plot_scaling_effect         trước và sau chuẩn hóa\n",
    "\"\"\"\n",
    "import numpy as np\n",
    "import pandas as pd\n",
    "import matplotlib.pyplot as plt\n",
    "from matplotlib.ticker import FuncFormatter, NullFormatter\n",
    "\n",
    "from .plotter import ClusterVisualizer, INK, NOISE_COLOR, PALETTE,
align_labels, make_snapshot, report_style\n",
    "\n",
    "BLUE, ORANGE, RED = PALETTE[0], PALETTE[1], PALETTE[3]\n",
    "_PLAIN = FuncFormatter(lambda v, _: f\"{v:g}\")\n",
    "\n",
    "def plot_scaling_effect(before, after, pair=(0, 1), names=None,
names_after=None, units=None):\n",
    "    \"\"\"So dữ liệu trước và sau chuẩn hóa (cùng số dòng, cùng thứ
tự cột). before, after: DataFrame hoặc mảng (n, d).\n",
    "\n",
    "    Trái: độ lệch chuẩn từng cột (thang log): trước chuẩn hóa một
cột có thể lấn át khoảng cách Euclid. Giữa và phải: tán xạ hai cột\n",
    "    `pair` trước và sau (names_after: tên cột sau chuẩn hóa, mặc
định như names). units: dict chỉ số cột -> (hệ số chia, tên đơn vị)
cho\n",
    "    panel trước chuẩn hóa, ví dụ {0: (1e6, \"triệu VND\")}, để trục
không hiện dạng 1e8. Trả Figure.\"\"\"\n",
    "    names = names or (list(before.columns) if hasattr(before,
\"columns\") else [f\"x{j + 1}\" for j in
range(np.asarray(before).shape[1])])\n",
    "    b, a = np.asarray(before, float), np.asarray(after, float)\n",
    "    if b.shape != a.shape:\n",
    "        raise ValueError(f\"before {b.shape} và after {a.shape} phải
cùng hình dạng\")\n",
```

```
    "    names_after = names_after or names\n",
    "    i, j = pair\n",
    "    with report_style():\n",
    "        fig, axes = plt.subplots(1, 3, figsize=(7.6, 3.0),
gridspec_kw={\"width_ratios\": [1.15, 1, 1]})\n",
    "        w = 0.38\n",
    "        x = np.arange(b.shape[1])\n",
    "        axes[0].bar(x - w / 2, b.std(axis=0), w, color=ORANGE,
label=\"Trước\")\n",
    "        axes[0].bar(x + w / 2, a.std(axis=0), w, color=BLUE,
label=\"Sau\")\n",
    "        axes[0].set_yscale(\"log\")\n",
    "        axes[0].set_xticks(x)\n",
    "        axes[0].set_xticklabels(names, rotation=35, ha=\"right\",
fontsize=6.5)\n",
    "        axes[0].set_ylabel(\"Độ lệch chuẩn (thang log)\")\n",
    "        axes[0].set_title(\"Thang đo từng cột\", loc=\"left\")\n",
    "        axes[0].legend(frameon=False)\n",
    "        units = units or {}\n",
    "        for ax, m, t, c, nm, u in ((axes[1], b, \"Trước chuẩn hóa\",
ORANGE, names, units), (axes[2], a, \"Sau chuẩn hóa\", BLUE, names_after,
{})):\n",
    "            fi, li = u.get(i, (1.0, None))\n",
    "            fj, lj = u.get(j, (1.0, None))\n",
    "            ax.scatter(m[:, i] / fi, m[:, j] / fj, s=3, color=c,
alpha=0.5, linewidths=0)\n",
    "            ax.set_xlabel(f\"{nm[i]} ({li})\" if li else nm[i],
fontsize=7)\n",
    "            ax.set_ylabel(f\"{nm[j]} ({lj})\" if lj else nm[j],
fontsize=7)\n",
    "            ax.set_title(t, loc=\"left\")\n",
    "        fig.tight_layout()\n",
    "    return fig\n"
   ]
  },
  {
   "cell_type": "markdown",
   "id": "4f2e504e",
   "metadata": {},
   "source": [
    "#### scenarios.py"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": null,
   "id": "8b6bedaf",
   "metadata": {
    "vscode": {
     "languageId": "plaintext"
    }
   },
   "outputs": [],
   "source": [
```

```
    "\"\"\"Đọc ví dụ cập nhật và stream rồi chạy trên MỘT engine bất kỳ
để lấy snapshot (không sklearn, không scipy).\n",
    "\n",
    "Engine được truyền vào dưới dạng hàm tạo `engine_factory(eps,
min_pts)` (RefitDBSCAN hôm nay, IncrementalDBSCAN của nhóm sau này), cần
các\n",
    "phương thức fit(data), insert(coords) -> id, delete(id) và một
snapshot lấy được bằng `snapshot_from_model`. Nhờ vậy mọi hình trong\n",
    "báo cáo đổi sang thuật toán thật chỉ bằng cách đổi đối số
`engine_factory`.\n",
    "\"\"\"\n",
    "import json\n",
    "from pathlib import Path\n",
    "\n",
    "import numpy as np\n",
    "import pandas as pd\n",
    "\n",
    "from src.preprocessing.stream_scaling import feature_columns,
stream_coordinates\n",
    "from .plotter import snapshot_from_model\n",
    "\n",
    "ROOT = Path(__file__).resolve().parents[2]\n",
    "RAW = ROOT / \"data\" / \"raw\"\n",
    "\n",
    "\n",
    "def list_scenarios(kind=\"incremental_examples\", raw_dir=RAW):\n",
    "    \"\"\"Tên các thư mục ví dụ cập nhật, theo thứ tự tên.\"\"\"\n",
    "    return sorted(p.name for p in (Path(raw_dir) / kind).iterdir()
if p.is_dir())\n",
    "\n",
    "\n",
    "def load_scenario(kind, name, raw_dir=RAW):\n",
    "    \"\"\"Đọc points.csv, update.csv, expected.json của một kịch
bản. Trả dict: names, coords, updates (list dict op, point_id,
coords),\n",
    "    eps, min_pts, expected, seed_ids (UpdSeed theo expected.json;
kịch bản nhiều bước lấy hợp các bước), coord_cols.\"\"\"\n",
    "    base = Path(raw_dir) / kind / name\n",
    "    exp = json.loads((base /
\"expected.json\").read_text(encoding=\"utf-8\"))\n",
    "    pts = pd.read_csv(base / \"points.csv\", dtype={\"point_id\":
str}, encoding=\"utf-8\")\n",
    "    upd = pd.read_csv(base / \"update.csv\", dtype={\"point_id\":
str}, encoding=\"utf-8\")\n",
    "    cols = [c for c in pts.columns if c != \"point_id\"]\n",
    "    updates = []\n",
    "    for _, r in upd.iterrows():\n",
    "        xy = None if r[\"op\"] == \"delete\" else
r[cols].to_numpy(dtype=float)\n",
    "        updates.append({\"op\": r[\"op\"], \"point_id\":
r[\"point_id\"], \"coords\": xy})\n",
    "    seeds = exp.get(\"updseed\")\n",
    "    if seeds is None:\n",
```

```
    "        seeds = sorted({s for step in exp.get(\"updseed_by_step\",
[]) for s in step})\n",
    "    return {\"kind\": kind, \"name\": name, \"names\":
pts[\"point_id\"].tolist(), \"coords\": pts[cols].to_numpy(dtype=float),
\"updates\": updates,\n",
    "            \"eps\": float(exp[\"eps\"]), \"min_pts\":
int(exp[\"minpts\"]), \"expected\": exp, \"seed_ids\": list(seeds),
\"coord_cols\": cols}\n",
    "\n",
    "\n",
    "def run_scenario(engine_factory, sc):\n",
    "    \"\"\"Chạy kịch bản trên engine: fit các điểm đầu, rồi từng dòng
update theo thứ tự.\n",
    "\n",
    "    Trả dict: before (snapshot sau fit), after (snapshot sau mọi
update), steps (snapshot sau từng update), update (danh sách để
truyền\n",
    "    cho plot_before_after), seed_ids, eps. Id trong snapshot là TÊN
điểm trong points.csv và update.csv (như A1, p, q), không phải id
engine.\"\"\"\n",
    "    eng = engine_factory(sc[\"eps\"], sc[\"min_pts\"])\n",
    "    eng.fit(sc[\"coords\"])\n",
    "    snap0 = snapshot_from_model(eng)\n",
    "    to_name = dict(zip(snap0[\"ids\"].tolist(), sc[\"names\"]))\n",
    "    to_engine = {n: e for e, n in to_name.items()}\n",
    "\n",
    "    def named(snap):\n",
    "        out = dict(snap)\n",
    "        out[\"ids\"] = np.array([to_name[i] for i in
snap[\"ids\"].tolist()])\n",
    "        return out\n",
    "    before, steps = named(snap0), []\n",
    "    for u in sc[\"updates\"]:\n",
    "        if u[\"op\"] == \"insert\":\n",
    "            eid = eng.insert(u[\"coords\"])\n",
    "            to_name[eid], to_engine[u[\"point_id\"]] =
u[\"point_id\"], eid\n",
    "        else:\n",
    "            eng.delete(to_engine.pop(u[\"point_id\"]))\n",
    "        steps.append(named(snapshot_from_model(eng)))\n",
    "    return {\"before\": before, \"after\": steps[-1] if steps else
before, \"steps\": steps, \"update\": sc[\"updates\"],\n",
    "            \"seed_ids\": sc[\"seed_ids\"], \"eps\":
sc[\"eps\"]}\n",
    "\n",
    "\n",
    "# ---------------------------------------------------------------
stream\n",
    "def stream_snapshots(engine_factory, stream_dir,
max_steps=None):\n",
    "    \"\"\"Chạy một stream (d0.csv, updates.csv, params.json) và trả
danh sách khung, mỗi khung ứng với một step (step 0 là sau fit D0).\n",
    "\n",
```

```
    "    Mỗi khung là dict: step, op ('fit', 'insert', 'delete'),
point_id (id của stream), coords (tọa độ điểm cập nhật hoặc None),\n",
    "    snapshot (ids là id của STREAM, coords theo stream_coordinates),
case (verified_case hoặc None), eps, min_pts.\"\"\"\n",
    "    stream_dir = Path(stream_dir)\n",
    "    d0 = pd.read_csv(stream_dir / \"d0.csv\", encoding=\"utf-
8\")\n",
    "    upd = pd.read_csv(stream_dir / \"updates.csv\", encoding=\"utf-
8\").sort_values(\"step\")\n",
    "    params = json.loads((stream_dir /
\"params.json\").read_text(encoding=\"utf-8\"))\n",
    "    feats = feature_columns(d0, params)\n",
    "    eps, min_pts = params.get(\"eps\", params.get(\"eps_km\")),
params[\"minpts\"]\n",
    "    eng = engine_factory(eps, min_pts)\n",
    "    eng.fit(stream_coordinates(params, d0, feats))\n",
    "    snap = snapshot_from_model(eng)\n",
    "    to_stream = dict(zip(snap[\"ids\"].tolist(),
d0[\"point_id\"].tolist()))\n",
    "    to_engine = {s: e for e, s in to_stream.items()}\n",
    "\n",
    "    def named(sn):\n",
    "        out = dict(sn)\n",
    "        out[\"ids\"] = np.array([to_stream[i] for i in
sn[\"ids\"].tolist()])\n",
    "        return out\n",
    "    frames = [{\"step\": 0, \"op\": \"fit\", \"point_id\": None,
\"coords\": None, \"snapshot\": named(snap), \"case\": None, \"eps\":
eps, \"min_pts\": min_pts}]\n",
    "    if max_steps is not None:\n",
    "        upd = upd[upd[\"step\"] <= max_steps]\n",
    "    coords = stream_coordinates(params, upd, feats)\n",
    "    for k, (_, r) in enumerate(upd.iterrows()):\n",
    "        sid = int(r[\"point_id\"])\n",
    "        if r[\"op\"] == \"insert\":\n",
    "            eid = eng.insert(coords[k])\n",
    "            to_stream[eid], to_engine[sid] = sid, eid\n",
    "        else:\n",
    "            eng.delete(to_engine.pop(sid))\n",
    "        frames.append({\"step\": int(r[\"step\"]), \"op\":
r[\"op\"], \"point_id\": sid, \"coords\": coords[k], \"snapshot\":
named(snapshot_from_model(eng)),\n",
    "                       \"case\": r.get(\"verified_case\"), \"eps\":
eps, \"min_pts\": min_pts})\n",
    "    return frames\n"
   ]
  }
 ],
 "metadata": {
  "language_info": {
   "name": "python"
  }
 },
 "nbformat": 4,
```

```
 "nbformat_minor": 5
}
```

# N01 - INCREMENTALDBSCAN - CODE - ĐẠO VĂN

ORIGINALITY REPORT

| 3% | 3% | 1% | % |
|---|---|---|---|
| SIMILARITY INDEX | INTERNET SOURCES | PUBLICATIONS | STUDENT PAPERS |

MATCH ALL SOURCES (ONLY SELECTED SOURCE PRINTED)

4%

★ cdnjs.cloudflare.com

Internet Source

| | | | |
|---|---|---|---|
| Exclude quotes | On | Exclude matches | < 100 words |
| Exclude bibliography | On | | |